

GUIA DE ESTUDO

Divisão Passo a Passo

O código do site explicado linha por linha

HTML, CSS e JavaScript, com os conceitos básicos, o funcionamento do programa, exercícios e glossário.

Arquivos explicados: `index.html` (70 linhas), `style.css` (210 linhas) e `script.js` (420 linhas).

Código-fonte: github.com/marcioalbinods/Testes

Outubro de 2026

Sumário

- Como usar este guia
- Parte 1: Os conceitos básicos (HTML, CSS e JavaScript)
- Parte 2: Como o programa pensa
- Parte 3: index.html, linha por linha
- Parte 4: style.css, linha por linha
- Parte 5: script.js, linha por linha
- Parte 6: Mãos à obra (como testar, exercícios, próximos passos e glossário)

Como usar este guia

Este guia explica, linha por linha, todo o código do site **Divisão Passo a Passo**. Ele foi escrito para quem está começando: não é preciso saber programar para acompanhar.

O site é formado por três arquivos, e cada um tem um papel:

Arquivo	Linhas	Tecnologia	Papel
index.html	70	HTML	A estrutura: o que existe na página.
style.css	210	CSS	A aparência: cores, tamanhos, posições.
script.js	420	JavaScript	O comportamento: contas, conferências, cliques.

O guia está dividido assim:

- **Parte 1:** os conceitos básicos de HTML, CSS e JavaScript, só o necessário para entender o código.
- **Parte 2:** como o programa "pensa", seguindo uma divisão de verdade do começo ao fim.
- **Partes 3, 4 e 5:** cada linha dos três arquivos, com o número da linha, o código e a explicação.
- **Parte 6:** exercícios para você mudar o site e aprender fazendo, mais um glossário.

Dica de estudo: abra o arquivo de código ao lado deste guia (no GitHub ou num editor como o VS Code). Os números das linhas do guia são os mesmos do arquivo. Leia um trecho, faça uma pequena mudança, veja o que acontece. Ninguém aprende a programar só lendo; é mexendo que se aprende.

Linhas em branco foram omitidas das tabelas: elas servem só para separar os trechos e deixar o código mais fácil de ler.

Parte 1: Os conceitos básicos

Como um site funciona

Quando você abre um site, o navegador (Chrome, Safari, Edge...) baixa arquivos de texto e os interpreta. Uma boa comparação é com uma casa:

- O **HTML** é a estrutura: paredes, portas e janelas. Ele diz **o que existe** na página (um título, um campo, um botão).
- O **CSS** é a decoração: pintura, móveis e cortinas. Ele diz **como cada coisa aparece** (cor, tamanho, posição).
- O **JavaScript** é a parte elétrica: interruptores e campainha. Ele diz **o que acontece** quando você interage (clica, escolhe, digita).

Nosso site não precisa de servidor nem de banco de dados. Tudo roda dentro do navegador de quem acessa, e é por isso que ele pode ser hospedado de graça no GitHub Pages.

HTML em poucos minutos

O HTML é feito de **tags** (etiquetas) entre os sinais `<` e `>`. A maioria das tags tem abertura e fechamento, e o fechamento tem uma barra:

```
<p>Isto é um parágrafo.</p>
<button id="btnComecar" class="primary">Começar</button>
```

- `<p>` abre e `</p>` fecha um parágrafo. O que está entre as duas tags é o conteúdo.
- `id="btnComecar"` e `class="primary"` são **atributos**: informações extras sobre o elemento, no formato `nome="valor"`.
- O **id** é um nome **único**, como um CPF: só um elemento da página pode ter aquele id. O JavaScript usa o id para encontrar o elemento.
- A **classe** é uma etiqueta que **vários** elementos podem compartilhar. O CSS usa as classes para aplicar estilos.
- As tags podem ficar umas dentro das outras, como caixas dentro de caixas. Uma tag fecha sempre antes da tag que a contém.

CSS em poucos minutos

O CSS é uma lista de **regras**. Cada regra diz: "para estes elementos, use estas características".

```
.card {
  background: white;
  border-radius: 14px;
}
```

- `.card` é o **seletor**: diz a quais elementos a regra se aplica.
- `background` e `border-radius` são **propriedades**; `white` e `14px` são os **valores**.
- Cada linha termina com ponto e vírgula, e o bloco fica entre chaves.

Os seletores mais usados no nosso código:

Seletor	Significa	Exemplo
nome da tag	todos os elementos dessa tag	button
.nome	elementos com essa classe	.card
#nome	o elemento com esse id	#conta
A B (com espaço)	B que está dentro de A	header h1
A.B (sem espaço)	elemento que tem A e B ao mesmo tempo	button.primary

Seletor	Significa	Exemplo
:hover, :focus	um estado (mouse em cima, selecionado)	button:hover

As unidades de medida mais usadas:

- **px**: pixels, os pontinhos da tela.
- **rem**: o tamanho da letra padrão da página (normalmente 16 px). Então **2rem** = 32 px.
- **em**: o tamanho da letra do próprio elemento.
- **vw**: 1% da largura da tela. Então **7vw** = 7% da largura.
- **%**: uma porcentagem do elemento que contém este.

Todo elemento é uma **caixa** com três camadas em volta do conteúdo: o **padding** (espaço interno), a **border** (borda) e a **margin** (espaço externo, que afasta os vizinhos).

Para posicionar as caixas, usamos dois sistemas:

- **Flex** (**display: flex**): coloca os filhos em **uma linha**, lado a lado.
- **Grid** (**display: grid**): coloca os filhos numa **grade** de linhas e colunas, como uma tabela. A conta da divisão é desenhada numa grade.

Por fim, as **variáveis CSS** (**--accent: #2f6fdb;**) guardam um valor com um nome, que depois é lido com **var(--accent)**. Assim, para trocar a cor do site inteiro, basta mudar um lugar.

JavaScript em poucos minutos

O JavaScript é uma linguagem de programação de verdade: tem variáveis, contas, decisões, repetições e funções.

Variáveis: caixas com nome

```
const divisor = 6;      // const: o valor não muda
let quociente = "";     // let: o valor pode mudar
quociente = quociente + "1";
```

Tipos de valor

Tipo	Exemplo	Para que serve
Texto (string)	"7854" ou 'olá'	Palavras e algarismos tratados como texto.
Número (number)	42, 3.14	Contas comuns.
BigInt	7854n	Inteiros enormes, com contas exatas. O n no final indica BigInt.
Booleano	true, false	Verdadeiro ou falso.
Vazio	null, undefined	"Nada": null é vazio de propósito; undefined é "não definido".
Lista (array)	[7, 8, 5, 4]	Vários valores em ordem. As posições começam em 0.
Objeto	{ nome: "Ana", idade: 9 }	Vários valores com nome.

Comparações e operadores

- **===** (igual) e **!==** (diferente); **>**, **<**, **>=**, **<=** comparam tamanhos.
- **&&** quer dizer **e**, **||** quer dizer **ou** e **!** quer dizer **não**.
- **+=** acrescenta ao valor e **++** soma 1.
- **condição ? a : b** é o **ternário**: "se a condição for verdadeira, a; senão, b".

Decisões e repetições

```
if (produto > P) {
  // grande demais
} else if (diferenca >= divisor) {
  // pequeno demais
} else {
  // certo!
}

for (let k = 0; k <= 9; k++) {
  // repete com k = 0, 1, 2, ... 9
}
```

Funções: receitas com nome

Uma função é um bloco de código que só roda quando é **chamada**. Ela pode receber informações (os **parâmetros**) e devolver um resultado com **return**.

```
function dobro(x) {
  return x * 2;
}
dobro(5); // resultado: 10

const dobro2 = (x) => x * 2; // a mesma coisa, no formato "função seta"
```

Mexendo na página: o DOM

O navegador transforma o HTML em objetos que o JavaScript pode ler e alterar. Esse conjunto de objetos se chama **DOM**. Os comandos que mais usamos:

Comando	O que faz
document.querySelector("#id")	Encontra um elemento da página.
document.createElement("div")	Cria um elemento novo (ainda invisível).
pai.appendChild(filho)	Coloca o elemento dentro de outro, e aí ele aparece.
el.textContent = "texto"	Troca o texto do elemento.
el.className = "a b"	Define as classes do elemento.
el.hidden = true	Esconde o elemento.
el.addEventListener("click", f)	Executa a função f quando o elemento for clicado.

Eventos: reagindo ao usuário

Um programa de página fica "esperando" as ações do usuário. Cada ação é um **evento**: **click** (clique), **change** (mudou a seleção), **keydown** (apertou uma tecla). Com **addEventListener**, dizemos qual função deve rodar em cada evento.

Textos com valores: template strings

Textos entre **crases** podem conter contas dentro de **`${ }`**:

```
const d = 6, k = 3;
`${d} × ${k} = ${d * k}` // resultado: "6 × 3 = 18"
```

Parte 2: Como o programa pensa

A ideia central: estado e desenho

O programa guarda tudo o que precisa lembrar num único objeto, chamado **S** (de **estado**): o dividendo, o divisor, as linhas já escritas, o quociente até agora, a fase atual etc.

Toda ação do usuário segue o mesmo ciclo:

- 1. O usuário faz algo (escolhe um número, clica num botão).
- 2. Uma função **muda o estado** (por exemplo, acrescenta um algarismo ao quociente).
- 3. A função **render()** **redesenha a tela inteira** a partir do estado.

Assim, a tela nunca "esquece" de atualizar nada: ela é sempre um reflexo fiel dos dados.

As três fases

O campo **S.fase** diz em que momento a conta está, e o painel muda conforme a fase:

Fase	O que o painel mostra	Para onde vai
escolher	"Quantas vezes o 6 cabe em 45?" e a caixa de seleção	Depois da subtração, vai para "baixar" ou "fim".
baixar	O botão "Baixar o 5 ↓"	Volta para "escolher".
fim	Quociente, resto e prova real	A conta acabou.

A grade de colunas

A conta é desenhada numa grade, como a folha quadriculada do caderno. Cada algarismo do dividendo ocupa uma coluna, contada a partir de 0. A coluna "-1" (à esquerda de tudo) é reservada para o sinal de menos. Na grade do CSS, que começa a contar do 1, a coluna -1 vira a primeira coluna, e é por isso que a linha 214 do script.js soma 2.

Coluna	-1	0	1	2	3
Dividendo		7	8	5	4
Produto	-	6			
Diferença		1	8		

Cada linha guarda só três informações: em que **coluna começa** (**início**), quais são os **algarismos** (**texto**) e de que **tipo** ela é (**dividendo**, **produto** ou **diferença**). Para alinhar o produto pela direita, embaixo do último algarismo do dividendo parcial, o programa calcula: início = coluna final - quantidade de algarismos + 1.

Acompanhando $7854 \div 6$ do começo ao fim

Passo	Dividendo parcial	Escolha	Produto	Diferença	Quociente
1	7	1	6	1	1
baixa o 8	18				
2	18	3	18	0	13
baixa o 5	05 = 5				

Passo	Dividendo parcial	Escolha	Produto	Diferença	Quociente
3	5	0 (6 não cabe em 5)	—	—	130
baixa o 4	054 = 54				
4	54	9	54	0	1309

Resultado: quociente **1309** e resto **0**. Prova real: $6 \times 1309 + 0 = 7854$.

Como o programa sabe se a escolha está certa

Quando você escolhe um algarismo k , a função `escolher` faz duas contas: **produto** = **divisor** $\times$ k e **diferença** = **parcial** – **produto**. Depois faz dois testes:

- Se o **produto é maior que o parcial**, k é **grande demais** (não dá para subtrair).
- Se a **diferença é maior ou igual ao divisor**, k é **pequeno demais** (ainda cabe mais uma vez).
- Se não caiu em nenhum dos dois, k está **certo**: é o único algarismo que funciona.

O caminho de um clique

O que acontece, em ordem, quando você escolhe "3" na caixa de seleção:

1. O navegador dispara o evento `change` na caixa de seleção (linha 325 do `script.js`).
2. A função `escolher("3")` calcula o produto e a diferença e guarda tudo em `S.escolha` (linhas 111 a 137).
3. A função `render()` chama `desenharConta()`, que desenha a prévia do produto embaixo do parcial; depois `desenharChave()`, que mostra o 3 provisório no quociente; e por fim `desenharPainel()`, que mostra a mensagem e o botão "Subtrair".
4. Ao clicar em "Subtrair", a função `subtrair()` grava as linhas do produto e da diferença no estado e chama `render()` de novo.

Parte 3: index.html, linha por linha

O **index.html** é o esqueleto do site. Ele diz quais elementos existem na página: títulos, campos, botões e caixas vazias que o JavaScript vai preencher depois.

Cabeçalho do documento (head) (linhas 1 a 12)

Essas linhas preparam a página: dizem ao navegador o tipo de arquivo, o idioma, o título e quais arquivos de estilo carregar. Nada disso aparece diretamente na tela.

Nº	Código	Explicação
1	<code><!doctype html></code>	Avisa ao navegador que este arquivo é um documento HTML5 , a versão moderna do HTML. Deve ser sempre a primeira linha de qualquer página.
2	<code><html lang="pt-BR"></code>	Abre o elemento <code><html></code> , que envolve a página inteira. O atributo <code>lang="pt-BR"</code> diz que o idioma é português do Brasil, o que ajuda leitores de tela, tradutores e o corretor ortográfico.
3	<code><head></code>	Abre o <code><head></code> (cabeça): uma área com informações sobre a página, que não aparecem no conteúdo visível.
4	<code><meta charset="utf-8"></code>	Define a codificação de caracteres como UTF-8. É ela que permite usar acentos, cedilha e símbolos como ÷ sem aparecer letras estranhas.
5	<code><meta name="viewport" content="width=device-width, initial-scale=1"></code>	A meta viewport faz a página se adaptar à largura da tela do celular. Sem ela, o celular mostraria a versão de computador em miniatura.
6	<code><title>Divisão Passo a Passo</title></code>	O <code><title></code> é o texto que aparece na aba do navegador e nos favoritos.
7	<code><meta name="description" content="Resolva divisões pelo método da chave, escolhendo cada algarismo do quociente."></code>	Uma descrição curta da página. O Google e os aplicativos de mensagem usam esse texto quando alguém compartilha o link.
8-9	<code><link rel="preconnect" href="https://fonts.googleapis.com"> <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin></code>	O preconnect pede ao navegador para já abrir a conexão com os servidores do Google Fonts, para que as fontes carreguem mais rápido. O crossorigin é necessário porque os arquivos de fonte vêm de outro endereço (outro "domínio").
10	<code><link href="https://fonts.googleapis.com/css2?family=Nunito:wght@400;600;800&family=Roboto+Mono:wght@500;700&display=swap" rel="stylesheet"></code>	Carrega as fontes do Google Fonts: Nunito (pesos 400, 600 e 800, usada nos textos) e Roboto Mono (pesos 500 e 700, usada nos números). O <code>display=swap</code> mostra o texto com uma fonte reserva enquanto a fonte definitiva ainda está carregando.
11	<code><link rel="stylesheet" href="style.css"></code>	Liga o nosso arquivo de estilos, o style.css . É assim que o HTML "encontra" o CSS.
12	<code></head></code>	Fecha o <code><head></code> . Toda tag que abre (<code><head></code>) precisa de uma tag que fecha, com barra (<code></head></code>).

Topo da página (linhas 13 a 18)

Daqui em diante, tudo fica dentro do `<body>`, ou seja, é o que aparece na tela.

Nº	Código	Explicação
13	<code><body></code>	Abre o <code><body></code> (corpo): tudo o que está aqui dentro aparece na tela.
14	<code><header></code>	Abre o <code><header></code> , o cabeçalho visível da página. É uma tag semântica : o nome dela já diz para que serve.
15	<code><h1>Divisão Passo a Passo</h1></code>	O <code><h1></code> é o título principal. Existem títulos de <code>h1</code> (mais importante) até <code>h6</code> . O ideal é ter só um <code>h1</code> por página.
16	<code><p>Método da chave: escolha cada algarismo do quociente e veja o produto e a diferença.</p></code>	O <code><p></code> é um parágrafo de texto, aqui usado como subtítulo.
17	<code></header></code>	Fecha o <code><header></code> .

Formulário de entrada (linhas 19 a 40)

Os campos onde o usuário digita o dividendo e o divisor, a caixa de marcar e os botões.

Nº	Código	Explicação
19	<code><main></code>	Abre o <code><main></code> , que guarda o conteúdo principal da página.
20	<code><section class="card setup" aria-label="Dados da divisão"></code>	Abre uma <code><section></code> (seção) com duas classes , <code>card</code> e <code>setup</code> , separadas por espaço. Classes são "etiquetas" que o CSS usa para aplicar estilos. O <code>aria-label</code> descreve a seção para leitores de tela usados por pessoas com deficiência visual.
21	<code><div class="fields"></code>	Uma <code><div></code> é uma caixa genérica, usada para agrupar coisas. Esta agrupa os dois campos, e o CSS da classe <code>fields</code> coloca os dois lado a lado.
22	<code><label>Dividendo</code>	Abre um <code><label></code> (rótulo) com o texto "Dividendo". Como o campo fica dentro do rótulo, os dois ficam ligados: clicar no texto coloca o cursor no campo.

Nº	Código	Explicação
23	<code><input id="dividendo" inputmode="numeric" autocomplete="off" placeholder="ex.: 7854" maxlength="15"></code>	O <code><input></code> é o campo de digitação do dividendo. Cada atributo tem uma função: • <code>id="dividendo"</code>: um nome único, que o JavaScript usa para encontrar este campo. • <code>inputmode="numeric"</code>: faz o celular abrir o teclado de números. • <code>autocomplete="off"</code>: impede o navegador de sugerir valores digitados antes. • <code>placeholder</code>: o texto cinza de exemplo que some quando você digita. • <code>maxlength="15"</code>: no máximo 15 caracteres. Não colocamos <code>type="number"</code> de propósito: assim o próprio programa confere se só há algarismos e mostra uma mensagem clara.
24	<code></label></code>	Fecha o <code><label></code> do dividendo.
25	<code>÷</code>	Um <code></code> é uma caixinha que fica dentro da linha de texto. Este mostra o símbolo ÷, que é só decoração. O <code>aria-hidden="true"</code> esconde o símbolo dos leitores de tela, porque ele não traz informação nova.
26	<code><label>Divisor</code>	Abre o rótulo "Divisor", do mesmo jeito que na linha 22.
27	<code><input id="divisor" inputmode="numeric" autocomplete="off" placeholder="ex.: 6" maxlength="6"></code>	Campo do divisor, igual ao do dividendo, mas com no máximo 6 algarismos.
28	<code></label></code>	Fecha o <code><label></code> do divisor.
29	<code></div></code>	Fecha a <code><div class="fields"></code> aberta na linha 21.
30	<code><label class="check"></code>	Abre o rótulo da caixa de marcar, com a classe <code>check</code> .
31	<code><input type="checkbox" id="praticar"></code>	Um <code><input type="checkbox"></code> é uma caixinha de marcar. O <code>id="praticar"</code> permite ao JavaScript saber se ela está marcada.
32	<code>Quero fazer a subtração (diferença) sozinho</code>	O texto que aparece ao lado da caixinha.
33	<code></label></code>	Fecha o rótulo.
34	<code><div class="buttons"></code>	Uma <code><div></code> que agrupa os botões. A classe <code>buttons</code> coloca os botões em linha, com espaço entre eles.
35	<code><button id="btnComecar" class="primary">Começar</button></code>	O botão "Começar". O <code>id="btnComecar"</code> permite ao JavaScript saber quando ele é clicado. A classe <code>primary</code> deixa o botão azul e cheio, porque é a ação principal.
36	<code><button id="btnExemplo">Exemplo aleatório</button></code>	O botão "Exemplo aleatório", que sorteia uma conta.
37	<code></div></code>	Fecha a <code><div></code> dos botões.
38	<code><p id="erroInicial" class="feedback bad" hidden></p></code>	Um parágrafo para mensagens de erro. As classes <code>feedback bad</code> deixam o texto vermelho. O atributo <code>hidden</code> faz ele começar escondido : o JavaScript só mostra quando há erro.

Nº	Código	Explicação
39	<code></section></code>	Fecha a primeira <code><section></code> .

Área de trabalho (linhas 41 a 67)

É a parte onde a conta é desenhada. As caixas começam vazias, porque o JavaScript as preenche durante o uso.

Nº	Código	Explicação
41	<code><section id="area" class="card" hidden></code>	A segunda seção, com <code>id="area"</code> . O atributo <code>hidden</code> a mantém escondida até o usuário clicar em "Começar".
42	<code><div class="paper-wrap"></code>	A <code><div class="paper-wrap"></code> envolve a folha. Se a conta ficar mais larga que a tela do celular, aparece uma barra de rolagem só aqui dentro, e a página inteira não fica torta.
43	<code><div class="paper"></code>	A <code><div class="paper"></code> é a folha quadriculada. Ela coloca a conta (à esquerda) e a chave (à direita) lado a lado.
44	<code><div id="conta" class="conta" aria-live="polite"></div></code>	A <code><div id="conta"></code> é onde o JavaScript desenha o dividendo, os produtos e as diferenças. Começa vazia. O <code>aria-live="polite"</code> faz os leitores de tela avisarem quando o conteúdo muda.
45	<code><div class="chave"></code>	Abre a <code><div class="chave"></code> , o lado direito da conta.
46	<code><div id="divisorBox" class="divisor-box"></div></code>	A caixa onde o divisor é escrito. É ela que recebe as linhas da "chave".
47	<code><div id="quocienteBox" class="quociente-box"></div></code>	A caixa onde vão aparecendo os algarismos do quociente.
48–50	<code></div> </div> </div></code>	Fecham, em ordem, a chave, a folha e o envoltório. As tags fecham na ordem inversa em que abriram, como caixas umas dentro das outras.
52	<code><div id="painel" class="painel"></div></code>	O painel de controle: pergunta, caixa de seleção, mensagens e botões. Fica vazio no HTML, porque o JavaScript o monta a cada passo.
54	<code><details class="tabuada"></code>	O <code><details></code> cria uma área que abre e fecha com um clique, sem precisar de JavaScript.
55	<code><summary>Ver a tabuada do divisor</summary></code>	O <code><summary></code> é o texto clicável que fica sempre visível no <code><details></code> .
56	<code><ul id="tabuada"></code>	Uma <code></code> (lista não numerada), onde o JavaScript coloca a tabuada do divisor.
57	<code></details></code>	Fecha o <code><details></code> .
59	<code><h2>Passos</h2></code>	Um subtítulo <code><h2></code> com o texto "Passos".
60	<code><ol id="passos" class="passos"></code>	Uma <code></code> (lista numerada). Cada item recebe o número 1, 2, 3... automaticamente. Aqui o JavaScript escreve cada passo da divisão.

Nº	Código	Explicação
62–64	<pre><div class="buttons"> <button id="btnRecomecar">Nova divisão</button> </div></pre>	Uma caixa com o botão "Nova divisão", que limpa tudo para começar outra conta.
65	<pre></section></pre>	Fecha a seção da área de trabalho.
66	<pre></main></pre>	Fecha o <code><main></code> .

Carregando o JavaScript (linhas 68 a 70)

Nº	Código	Explicação
68	<pre><script src="script.js"></script></pre>	Carrega o arquivo <code>script.js</code> . Ele fica no final do <code><body></code> de propósito: assim, quando o código roda, todos os elementos da página já existem e podem ser encontrados.
69	<pre></body></pre>	Fecha o <code><body></code> .
70	<pre></html></pre>	Fecha o <code><html></code> . Fim do arquivo.

Parte 4: style.css, linha por linha

O `style.css` define a aparência do site: cores, tamanhos, a folha quadriculada, o desenho da chave e os traços da subtração.

Algumas linhas curtas reúnem várias propriedades numa linha só, como `.check input { width: 1.1rem; height: 1.1rem; }`. Funciona do mesmo jeito que escrever uma propriedade por linha.

Variáveis de cores e tamanhos (linhas 1 a 16)

Aqui guardamos as cores e o tamanho dos quadradinhos em **variáveis**. Se você quiser mudar a cor azul do site inteiro, basta mudar uma linha.

Nº	Código	Explicação
1	<code>:root {</code>	<code>:root</code> seleciona a "raiz" do documento, o elemento <code><html></code> . É o lugar costumeiro para criar variáveis CSS , que valem para a página toda. A chave <code>{</code> abre o bloco de regras.
2	<code>--bg: #f4f1ea;</code>	Cria a variável <code>--bg</code> (de <i>*background*</i> , fundo), com a cor bege claro do fundo da página. Variáveis CSS sempre começam com dois tracinhos. A cor está em hexadecimal , no formato <code>#RRGGBB</code> : duas letras para vermelho, duas para verde e duas para azul, de <code>00</code> (nada) até <code>ff</code> (máximo).
3	<code>--card: #ffffff;</code>	<code>--card</code> : branco puro (<code>#ffffff</code>), usado no fundo dos cartões.
4	<code>--ink: #1f2a44;</code>	<code>--ink</code> ("tinta"): o azul-marinho bem escuro usado nos textos e nos traços da conta.
5	<code>--muted: #5b6478;</code>	<code>--muted</code> ("apagado"): um cinza azulado para textos menos importantes.
6	<code>--line: #d9e3f0;</code>	<code>--line</code> : o azul bem clarinho das linhas da folha quadriculada e das bordas.
7	<code>--accent: #2f6fdb;</code>	<code>--accent</code> ("destaque"): o azul forte dos botões e do produto.
8	<code>--accent-soft: #e3edff;</code>	<code>--accent-soft</code> : a versão bem clara do azul, usada como fundo do painel.
9	<code>--good: #1d8a4e;</code>	<code>--good</code> : verde para os acertos.
10	<code>--good-soft: #e2f5ea;</code>	<code>--good-soft</code> : verde clarinho, usado no fundo das mensagens de acerto.
11	<code>--bad: #c0392b;</code>	<code>--bad</code> : vermelho para os erros.
12	<code>--bad-soft: #fde8e6;</code>	<code>--bad-soft</code> : vermelho clarinho, usado no fundo das mensagens de erro.
13	<code>--hl: #fff2b3;</code>	<code>--hl</code> (de <i>*highlight*</i>): o amarelo "marca-texto" que destaca o dividendo parcial.

Nº	Código	Explicação
14	<code>--cell: clamp(1.9rem, 7vw, 2.6rem);</code>	<code>--cell</code> é o tamanho de cada quadradinho da folha. A função <code>clamp(mínimo, ideal, máximo)</code> escolhe um valor entre os limites. O ideal é <code>7vw</code> , ou 7% da largura da tela, mas nunca menos que <code>1.9rem</code> nem mais que <code>2.6rem</code> . A unidade <code>rem</code> vale o tamanho da letra padrão, normalmente 16 pixels. Assim, os quadradinhos se ajustam ao celular e ao computador.
15	<code>}</code>	Fecha o bloco de regras de <code>:root</code> , aberto na linha 1.

Estilos gerais da página (linhas 17 a 50)

Nº	Código	Explicação
17	<code>* { box-sizing: border-box; }</code>	O asterisco <code>*</code> seleciona todos os elementos. O <code>box-sizing: border-box</code> faz a largura de cada elemento já incluir o espaço interno e a borda, o que deixa os tamanhos mais fáceis de calcular.
19	<code>body {</code>	Abre as regras do <code>body</code> , que valem para toda a parte visível.
20	<code>margin: 0;</code>	<code>margin: 0</code> remove a margem que o navegador coloca por padrão em volta da página.
21	<code>background: var(--bg);</code>	<code>background</code> define o fundo. A função <code>var(--bg)</code> lê a variável criada na linha 2.
22	<code>color: var(--ink);</code>	A cor padrão do texto, lida da variável <code>--ink</code> .
23	<code>font-family: "Nunito", system-ui, sans-serif;</code>	A fonte do texto. É uma lista de opções em ordem de preferência: primeiro a Nunito; se ela não carregar, a fonte do sistema (<code>system-ui</code>); e por último qualquer fonte sem serifa (<code>sans-serif</code>).
24	<code>line-height: 1.5;</code>	<code>line-height: 1.5</code> é o espaço entre as linhas do texto: 1,5 vez o tamanho da letra, o que facilita a leitura.
25	<code>}</code>	Fecha o bloco de regras de <code>body</code> , aberto na linha 19.
27	<code>header {</code>	Abre as regras do <code><header></code> .
28	<code>text-align: center;</code>	Centraliza o texto do cabeçalho.
29	<code>padding: 1.5rem 16px 0.5rem;</code>	<code>padding</code> é o espaço interno . Com três valores, a ordem é: em cima (<code>1.5rem</code>), nos lados (<code>16px</code>) e embaixo (<code>0.5rem</code>).
30	<code>}</code>	Fecha o bloco de regras de <code>header</code> , aberto na linha 27.
31	<code>header h1 { margin: 0; font-size: clamp(1.6rem, 5vw, 2.2rem); font-weight: 800; }</code>	<code>header h1</code> (com espaço entre os nomes) seleciona o <code>h1</code> que está dentro de um <code>header</code> . Essa regra tira a margem, define um tamanho flexível com <code>clamp</code> e usa o peso 800 (extranegrito).

Nº	Código	Explicação
32	header p { margin: 0.25rem 0 0; color: var(--muted); }	O parágrafo do cabeçalho. margin: 0.25rem 0 0 coloca margem só em cima, e a cor vem da variável --muted .
34	main {	Abre as regras do <main> , o conteúdo principal.
35	max-width: 820px;	max-width: 820px : o conteúdo nunca passa de 820 pixels de largura, para não ficar esticado em telas grandes.
36	margin: 0 auto;	margin: 0 auto deixa a margem de cima e de baixo em zero e a dos lados automática. Isso centraliza o bloco na tela.
37	padding: 1rem 16px 3rem;	Espaço interno: em cima, nos lados e embaixo.
38	display: grid;	display: grid transforma o elemento numa grade . Aqui, os filhos ficam um embaixo do outro, em uma única coluna.
39	gap: 1rem;	gap é o espaço entre os filhos da grade.
40	}	Fecha o bloco de regras de main , aberto na linha 34.
42	.card {	Abre as regras da classe .card . O ponto antes do nome indica que é uma classe, e não uma tag.
43	background: var(--card);	Fundo branco, lido da variável --card .
44	border-radius: 14px;	border-radius arredonda os cantos, aqui com raio de 14 pixels.
45	padding: 1.25rem;	Espaço interno de 1.25rem em todos os lados.
46	box-shadow: 0 2px 10px rgba(31, 42, 68, 0.08);	box-shadow cria uma sombra. Os valores são: deslocamento horizontal 0 , deslocamento vertical 2px , desfoque 10px e a cor rgba(. . .) . O último número do rgba , 0.08 , é a opacidade: 8%, uma sombra bem suave.
47	}	Fecha o bloco de regras de .card , aberto na linha 42.
49	h2 { font-size: 1.1rem; margin: 1.25rem 0 0.5rem; }	Estilo dos subtítulos h2 : tamanho da letra e margens.

Campos de entrada e botões (linhas 51 a 102)

Nº	Código	Explicação
51	/* ----- Entrada ----- */	Um comentário de CSS fica entre /* e */ . O navegador ignora o que está ali; ele serve só para organizar o arquivo.
52	.fields {	Abre as regras da caixa .fields , que agrupa os campos.
53	display: flex;	display: flex coloca os filhos lado a lado , numa linha.

Nº	Código	Explicação
54	<code>flex-wrap: wrap;</code>	flex-wrap: wrap : se não couberem lado a lado, os filhos "quebram" para a linha de baixo, como acontece no celular.
55	<code>align-items: flex-end;</code>	align-items: flex-end alinha os filhos pela parte de baixo.
56	<code>gap: 0.75rem;</code>	O espaço entre os filhos.
57	<code>}</code>	Fecha o bloco de regras de .fields , aberto na linha 52.
58	<code>.fields label {</code>	Regras para os label dentro de .fields .
59	<code>display: grid;</code>	display: grid faz o texto do rótulo ficar em cima e o campo embaixo.
60	<code>gap: 0.25rem;</code>	O espaço entre o texto e o campo.
61	<code>font-weight: 600;</code>	font-weight: 600 deixa o texto em seminegrito.
62	<code>flex: 1 1 140px;</code>	flex: 1 1 140px é um atalho para três valores. O primeiro 1 deixa o campo crescer para ocupar o espaço livre. O segundo 1 deixa o campo encolher se faltar espaço. E 140px é o tamanho base.
63	<code>}</code>	Fecha o bloco de regras de .fields label , aberto na linha 58.
64	<code>.div-sign { font-size: 1.8rem; font-weight: 800; padding-bottom: 0.2rem; color: var(--muted); }</code>	O símbolo ÷: tamanho grande, extranegrito, um pequeno espaço embaixo e cor apagada.
66	<code>input[type="text"], input:not([type]) {</code>	Esse seletor pega os campos de texto: input[type="text"] são os campos com type="text" , e input:not([type]) são os campos que não têm o atributo type , como os nossos. A vírgula aplica as mesmas regras aos dois.
67	<code>font: 700 1.3rem "Roboto Mono", monospace;</code>	font é um atalho que define, de uma vez, o peso (700 , negrito), o tamanho (1.3rem) e a fonte (Roboto Mono). Usamos uma fonte monoespaçada , em que todo caractere tem a mesma largura, para os algarismos ficarem alinhados.
68	<code>padding: 0.5rem 0.7rem;</code>	O espaço interno do campo.
69	<code>border: 2px solid var(--line);</code>	A borda: 2 pixels, linha sólida (solid), na cor --line .
70	<code>border-radius: 10px;</code>	Os cantos arredondados do campo.
71	<code>width: 100%;</code>	width: 100% faz o campo ocupar toda a largura disponível.
72	<code>color: var(--ink);</code>	A cor do texto digitado.
73	<code>}</code>	Fecha o bloco de regras de input[type="text"], input:not([type]) , aberto na linha 66.

Nº	Código	Explicação
74	<code>input:focus, select:focus, button:focus-visible {</code>	As pseudoclasses <code>:focus</code> e <code>:focus-visible</code> descrevem um estado do elemento: aqui, quando ele está selecionado (com o cursor dentro ou escolhido pelo teclado). Mostrar bem esse estado é importante para quem navega usando só o teclado.
75	<code>outline: 3px solid var(--accent-soft);</code>	Quando o elemento está selecionado, ganha um contorno (outline) azul-claro de 3 pixels.
76	<code>border-color: var(--accent);</code>	E a borda fica azul forte.
77	<code>}</code>	Fecha o bloco de regras de <code>input:focus</code> , <code>select:focus</code> , <code>button:focus-visible</code> , aberto na linha 74.
79	<code>.check {</code>	Abre as regras da linha da caixa de marcar.
80–82	<code>display: flex; gap: 0.5rem; align-items: center;</code>	flex coloca a caixinha e o texto lado a lado, com gap de espaço entre eles, e align-items: center centraliza os dois na vertical.
83	<code>margin-top: 0.9rem;</code>	Uma margem em cima, para separar dos campos.
84	<code>cursor: pointer;</code>	cursor: pointer mostra a "mãozinha" quando o mouse passa por cima, indicando que dá para clicar.
85	<code>}</code>	Fecha o bloco de regras de <code>.check</code> , aberto na linha 79.
86	<code>.check input { width: 1.1rem; height: 1.1rem; }</code>	O tamanho da caixinha de marcar.
88	<code>.buttons { display: flex; flex-wrap: wrap; gap: 0.6rem; margin-top: 1rem; }</code>	A caixa dos botões: botões lado a lado, que quebram de linha se precisarem, com espaço entre eles e uma margem em cima.
90	<code>button {</code>	Abre as regras de todos os botões.
91	<code>font: 700 1rem "Nunito", sans-serif;</code>	A fonte do botão: negrito, tamanho 1rem , família Nunito.
92	<code>padding: 0.6rem 1.1rem;</code>	O espaço interno: 0.6rem em cima e embaixo, 1.1rem nos lados.
93	<code>border-radius: 10px;</code>	Os cantos arredondados.
94	<code>border: 2px solid var(--accent);</code>	Uma borda azul de 2 pixels.
95	<code>background: var(--card);</code>	O fundo branco.
96	<code>color: var(--accent);</code>	O texto azul.
97	<code>cursor: pointer;</code>	A mãozinha do mouse.
98	<code>}</code>	Fecha o bloco de regras de <code>button</code> , aberto na linha 90.
99	<code>button:hover { background: var(--accent-soft); }</code>	:hover é o estado de quando o mouse passa por cima . Nesse momento, o fundo fica azul-claro.

Nº	Código	Explicação
100	<code>button.primary { background: var(--accent); color: #fff; }</code>	<code>button.primary</code> , sem espaço , quer dizer "botão que também tem a classe primary". Esse botão tem fundo azul e texto branco.
101	<code>button.primary:hover { filter: brightness(1.08); }</code>	Quando o mouse passa sobre o botão principal, <code>filter: brightness(1.08)</code> deixa ele 8% mais claro.

Folha quadriculada e conta (linhas 103 a 153)

Nº	Código	Explicação
103	<code>/* ----- Folha quadriculada ----- */</code>	Um comentário que marca a seção da folha quadriculada.
104	<code>.paper-wrap { overflow-x: auto; }</code>	<code>overflow-x: auto</code> : se o conteúdo for mais largo que a caixa, aparece uma barra de rolagem horizontal , e só nesse caso.
105	<code>.paper {</code>	Abre as regras da folha (<code>.paper</code>).
106	<code>display: inline-flex;</code>	<code>inline-flex</code> funciona como o <code>flex</code> , mas a caixa ocupa só o tamanho do próprio conteúdo.
107	<code>align-items: flex-start;</code>	Alinha a conta e a chave pelo topo.
108	<code>gap: var(--cell);</code>	O espaço entre a conta e a chave é exatamente um quadradinho . Assim, tudo continua encaixado na grade.
109	<code>padding: var(--cell);</code>	A margem interna também tem um quadradinho.
110	<code>min-width: 100%;</code>	<code>min-width: 100%</code> : a folha ocupa no mínimo toda a largura do cartão.
111	<code>background-color: #fbfdff;</code>	A cor de fundo do papel, um branco levemente azulado.
112–114	<code>background-image: linear-gradient(var(--line) 1px, transparent 1px), linear-gradient(90deg, var(--line) 1px, transparent 1px);</code>	É aqui que nasce o quadriculado . O <code>background-image</code> recebe duas imagens geradas pelo próprio CSS com <code>linear-gradient</code> (degradê). A primeira pinta 1 pixel com a cor da linha e o resto transparente, o que forma uma linha horizontal . A segunda, girada 90deg (90 graus), forma uma linha vertical . Repetidas, as duas formam os quadradinhos.
115	<code>background-size: var(--cell) var(--cell);</code>	<code>background-size</code> faz cada repetição ter o tamanho de um quadradinho (<code>--cell</code>). Por isso as linhas do papel coincidem com as células da conta.
116	<code>border-radius: 10px;</code>	Os cantos arredondados da folha.
117	<code>border: 1px solid var(--line);</code>	Uma borda fina em volta da folha.
118	<code>}</code>	Fecha o bloco de regras de <code>.paper</code> , aberto na linha 105.
120	<code>.conta {</code>	Abre as regras da grade da conta (<code>#conta</code> , que tem a classe <code>conta</code>).

Nº	Código	Explicação
121	<code>display: grid;</code>	Transforma a conta numa grade (grid). Cada algarismo vai ocupar uma posição de linha e coluna.
122	<code>grid-auto-columns: var(--cell);</code>	grid-auto-columns : toda coluna criada tem a largura de um quadradinho.
123	<code>grid-auto-rows: var(--cell);</code>	grid-auto-rows : toda linha criada tem a altura de um quadradinho.
124	<code>}</code>	Fecha o bloco de regras de .conta , aberto na linha 120.
126	<code>.cell, .chave span {</code>	Regras comuns às células da conta (.cell) e aos algarismos da chave (.chave span).
127– 128	<code>width: var(--cell); height: var(--cell);</code>	A largura e a altura são iguais a um quadradinho.
129– 131	<code>display: flex; align-items: center; justify-content: center;</code>	display: flex com align-items: center e justify-content: center coloca o algarismo bem no centro do quadradinho, na vertical e na horizontal.
132	<code>font: 700 calc(var(--cell) * 0.6) "Roboto Mono", monospace;</code>	A fonte dos algarismos. calc(var(--cell) * 0.6) faz uma conta dentro do CSS: o tamanho da letra é 60% do tamanho do quadradinho.
133	<code>}</code>	Fecha o bloco de regras de .cell, .chave span , aberto na linha 126.
135	<code>.cell.partial { background: var(--hl); }</code>	.cell.partial , com as duas classes juntas, é a célula do dividendo parcial. Ela ganha o fundo amarelo (marca-texto).
136	<code>.cell.used { color: var(--muted); }</code>	.used : o algarismo do dividendo que já foi "baixado" fica cinza.
137	<code>.cell.down { position: relative; }</code>	position: relative transforma a célula num ponto de referência para posicionar a setinha da próxima regra.
138	<code>.cell.down::after {</code>	::after (com dois pontos duas vezes) é um pseudoelemento : um conteúdo extra que o CSS cria depois do conteúdo da célula.
139	<code>content: "↓";</code>	content: "↓" : o conteúdo extra é uma setinha para baixo.
140	<code>position: absolute;</code>	position: absolute posiciona a setinha em relação à célula, que é o ponto de referência da linha 137.
141– 142	<code>top: -0.15rem; right: 0.05rem;</code>	Coloca a setinha no canto de cima, à direita.
143	<code>font-size: 0.6em;</code>	0.6em : a setinha tem 60% do tamanho da letra da célula. A unidade em é relativa à letra do próprio elemento.
144	<code>color: var(--accent);</code>	A setinha é azul.
145	<code>}</code>	Fecha o bloco de regras de .cell.down::after , aberto na linha 138.

Nº	Código	Explicação
146	<code>.cell.ul { box-shadow: inset 0 -3px 0 var(--ink); }</code>	<code>.ul</code> (de *underline*, sublinhado) é o traço da subtração . O truque usa <code>box-shadow</code> com <code>inset</code> (sombra interna) deslocada <code>-3px</code> para cima e sem desfoque. O resultado é uma linha de 3 pixels na base da célula, que não muda o tamanho dela.
147	<code>.cell.product { color: var(--accent); }</code>	O produto fica azul.
148	<code>.cell.diff { color: var(--good); }</code>	A diferença (o resultado da subtração) fica verde.
149	<code>.cell.brought { color: var(--ink); }</code>	O algarismo que foi "baixado" fica na cor normal do texto.
150	<code>.cell.preview { color: var(--accent); opacity: 0.85; }</code>	<code>.preview</code> é a prévia do produto, mostrada enquanto você escolhe. Ela fica azul e um pouco transparente: <code>opacity: 0.85</code> é 85% visível.
151	<code>.cell.preview.wrong { color: var(--bad); }</code>	Se a escolha estiver errada (classe <code>wrong</code>), a prévia fica vermelha.
152	<code>.cell.preview.ul.wrong { box-shadow: inset 0 -3px 0 var(--bad); }</code>	E o traço da prévia errada também fica vermelho.

A chave (divisor e quociente) (linhas 154 a 165)

Nº	Código	Explicação
154	<code>.chave { display: grid; }</code>	A chave é uma grade: o divisor em cima e o quociente embaixo.
155	<code>.divisor-box, .quociente-box { display: flex; min-height: var(--cell); }</code>	O divisor e o quociente põem os algarismos lado a lado (<code>flex</code>), com altura mínima de um quadradinho.
156	<code>.divisor-box {</code>	Abre as regras da caixa do divisor.
157	<code>box-shadow: inset 3px 0 0 var(--ink), inset 0 -3px 0 var(--ink);</code>	É isto que desenha a chave . São duas sombras internas separadas por vírgula. <code>inset 3px 0 0</code> desenha uma linha de 3 pixels na esquerda (o traço vertical), e <code>inset 0 -3px 0</code> desenha uma linha de 3 pixels embaixo (o traço horizontal). Juntas, formam o "L" da chave.
158	<code>padding-right: calc(var(--cell) * 0.5);</code>	Um espaço à direita de meio quadradinho, para o traço horizontal passar um pouco do divisor.
159	<code>}</code>	Fecha o bloco de regras de <code>.divisor-box</code> , aberto na linha 156.
160	<code>.quociente-box { padding-left: 3px; }</code>	O quociente ganha um recuo de 3 pixels à esquerda, para ficar alinhado com o divisor, que tem o traço vertical de 3 pixels.
161	<code>.quociente-box span { color: var(--accent); }</code>	Os algarismos do quociente são azuis.
162	<code>.quociente-box span.temp { color: var(--muted); background: var(--accent-soft); border-radius: 6px; }</code>	<code>.temp</code> é o algarismo provisório : o que você está escolhendo e ainda não confirmou. Ele tem fundo azul-claro e cantos arredondados.

Nº	Código	Explicação
163	<code>.quociente-box span.temp.wrong { color: var(--bad); background: var(--bad-soft); }</code>	Se o algoritmo provisório estiver errado, fica vermelho, com fundo vermelho-claro.
164	<code>.quociente-box span.empty { color: var(--muted); }</code>	O ponto de interrogação ? , mostrado antes de você escolher, fica na cor apagada.

Painel de controle e mensagens (linhas 166 a 193)

Nº	Código	Explicação
166	<code>/* ----- Painel de controle ----- */</code>	Um comentário que marca a seção do painel.
167	<code>.painel {</code>	Abre as regras do painel.
168	<code>margin-top: 1rem;</code>	Uma margem em cima.
169	<code>padding: 1rem;</code>	Um espaço interno.
170	<code>border-radius: 12px;</code>	Os cantos arredondados.
171	<code>background: var(--accent-soft);</code>	O fundo azul-claro.
172	<code>}</code>	Fecha o bloco do painel.
173	<code>.painel .pergunta { font-size: 1.15rem; font-weight: 700; margin: 0 0 0.6rem; }</code>	A pergunta do painel ("Quantas vezes o 6 cabe em 45?"): letra um pouco maior, em negrito, com margem embaixo.
174	<code>.painel .linha { display: flex; flex-wrap: wrap; gap: 0.6rem; align-items: center; }</code>	A linha que mostra o rótulo e a caixa de seleção lado a lado.
175	<code>.painel select, .painel input {</code>	Regras comuns à caixa de seleção (select) e ao campo de diferença (input) do painel.
176	<code>font: 700 1.3rem "Roboto Mono", monospace;</code>	A fonte monoespaçada em negrito, num tamanho grande.
177	<code>padding: 0.35rem 0.6rem;</code>	O espaço interno.
178	<code>border: 2px solid var(--accent);</code>	Uma borda azul.
179	<code>border-radius: 10px;</code>	Os cantos arredondados.
180	<code>background: #fff;</code>	O fundo branco (#fff é o jeito curto de escrever #ffffff).
181	<code>color: var(--ink);</code>	A cor do texto.
182	<code>}</code>	Fecha o bloco de regras de .painel select , .painel input , aberto na linha 175.
183	<code>.painel input { width: 9rem; }</code>	A largura do campo onde se digita a diferença.
184	<code>.painel .buttons { margin-top: 0.8rem; }</code>	A margem acima dos botões do painel.
186	<code>.feedback { margin: 0.7rem 0 0; padding: 0.6rem 0.8rem; border-radius: 10px; font- weight: 600; }</code>	.feedback é o estilo comum a todas as mensagens: margem, espaço interno, cantos arredondados e seminegrito.

Nº	Código	Explicação
187	<code>.feedback.good { background: var(--good-soft); color: var(--good); }</code>	Mensagem de acerto (good): fundo verde-claro e texto verde.
188	<code>.feedback.bad { background: var(--bad-soft); color: var(--bad); }</code>	Mensagem de erro (bad): fundo vermelho-claro e texto vermelho.
189	<code>.feedback.info { background: #fff; color: var(--ink); }</code>	Mensagem informativa (info): fundo branco e texto normal.
191	<code>.resultado { font-size: 1.1rem; }</code>	A área do resultado final tem a letra um pouco maior.
192	<code>.resultado strong { font-family: "Roboto Mono", monospace; }</code>	Os números em negrito do resultado usam a fonte monoespçada.

Tabuada e passos (linhas 194 a 210)

Nº	Código	Explicação
194	<code>/* ----- Tabuada e passos ----- */</code>	Um comentário que marca a última seção.
195	<code>.tabuada { margin-top: 1rem; }</code>	Uma margem acima do quadro da tabuada.
196	<code>.tabuada summary { cursor: pointer; font-weight: 700; color: var(--accent); }</code>	O texto clicável "Ver a tabuada do divisor": mãozinha do mouse, negrito e azul.
197	<code>.tabuada ul {</code>	Abre as regras da lista da tabuada.
198	<code>list-style: none;</code>	list-style: none tira as bolinhas da lista.
199	<code>padding: 0;</code>	Tira o recuo padrão das listas.
200	<code>margin: 0.6rem 0 0;</code>	Uma margem só em cima.
201	<code>display: grid;</code>	A lista vira uma grade.
202	<code>grid-template-columns: repeat(auto-fill, minmax(8.5rem, 1fr));</code>	repeat(auto-fill, minmax(8.5rem, 1fr)) cria quantas colunas couberem , cada uma com pelo menos 8.5rem . A unidade 1fr ("uma fração") divide o espaço que sobra igualmente entre elas. Por isso há mais colunas no computador e menos no celular.
203	<code>gap: 0.35rem;</code>	O espaço entre os itens.
204	<code>font-family: "Roboto Mono", monospace;</code>	A fonte monoespçada.
205	<code>}</code>	Fecha o bloco de regras de .tabuada ul , aberto na linha 197.
206	<code>.tabuada li { background: var(--bg); border-radius: 8px; padding: 0.25rem 0.5rem; }</code>	Cada item da tabuada tem fundo bege, cantos arredondados e um espaço interno.
208	<code>.passos { margin: 0; padding-left: 1.4rem; }</code>	A lista de passos: sem margem e com um recuo à esquerda para os números.
209	<code>.passos li { margin-bottom: 0.3rem; }</code>	Um espacinho embaixo de cada passo.

Nº	Código	Explicação
210	<code>.passos code { font-family: "Roboto Mono", monospace; font-weight: 700; }</code>	Os números destacados nos passos (a tag <code><code></code>) usam a fonte monoespçada em negrito.

Parte 5: script.js, linha por linha

O `script.js` é o cérebro do site, escrito em JavaScript. Ele confere os números digitados, faz as contas, decide se a escolha está certa e desenha a conta na tela.

Uma convenção usada no código: **D** (maiúsculo) é o dividendo, guardado como texto; **d** (minúsculo) é o divisor, guardado como número BigInt; **P** é o dividendo parcial; e **k** é o algarismo escolhido para o quociente.

Comentários iniciais e referências aos elementos (linhas 1 a 22)

Primeiro, o programa "pega" os elementos da página que vai usar e os guarda num lugar fácil de acessar.

Nº	Código	Explicação
1-3	<pre>// Divisão passo a passo pelo método da chave. // A conta é desenhada numa grade: a coluna 0 guarda o sinal de menos e // a coluna c+1 guarda o algarismo c do dividendo.</pre>	Linhas que começam com <code>//</code> são comentários : o computador ignora o que vem depois delas. Servem para explicar o código a quem lê. Esses três comentários contam a ideia geral: a conta é desenhada numa grade, com a coluna 0 reservada para o sinal de menos.
5	<pre>const \$ = (sel) => document.querySelector(sel);</pre>	Cria uma constante (<code>const</code>) chamada <code>\$</code> , que guarda uma função . A forma <code>(sel) => ...</code> é uma <i>*arrow function*</i> (função seta): ela recebe um seletor (<code>sel</code>) e devolve <code>document.querySelector(sel)</code> , que é o primeiro elemento da página que combina com o seletor. É só um atalho para não escrever <code>document.querySelector</code> toda vez. O nome <code>\$</code> não tem nada de mágico: é um nome válido como qualquer outro.
7	<pre>const els = {</pre>	Cria a constante <code>els</code> (de "elementos"), que guarda um objeto . Um objeto é um conjunto de pares <code>nome: valor</code> , entre chaves <code>{ }</code> . Aqui vamos guardar os elementos da página que o programa usa muito, para buscá-los uma vez só.
8	<pre> dividendo: \$("#dividendo"),</pre>	Busca o elemento com <code>id="dividendo"</code> (o <code>#</code> no seletor significa "procure pelo id") e o guarda com o nome <code>dividendo</code> . Depois, basta escrever <code>els.dividendo</code> para usar o campo.
9	<pre> divisor: \$("#divisor"),</pre>	Guarda o campo do divisor em <code>els.divisor</code> .
10	<pre> praticar: \$("#praticar"),</pre>	Guarda a caixinha "Quero fazer a subtração sozinho" em <code>els.praticar</code> .
11	<pre> erroInicial: \$("#erroInicial"),</pre>	Guarda o parágrafo de mensagem de erro em <code>els.erroInicial</code> .
12	<pre> area: \$("#area"),</pre>	Guarda a área de trabalho em <code>els.area</code> .
13	<pre> conta: \$("#conta"),</pre>	Guarda a grade onde a conta é desenhada em <code>els.conta</code> .
14	<pre> divisorBox: \$("#divisorBox"),</pre>	Guarda a caixa do divisor, na chave, em <code>els.divisorBox</code> .
15	<pre> quocienteBox: \$("#quocienteBox"),</pre>	Guarda a caixa do quociente em <code>els.quocienteBox</code> .

Nº	Código	Explicação
16	<code>painel: \$("#painel"),</code>	Guarda o painel de controle em <code>els.painel</code> .
17	<code>tabuada: \$("#tabuada"),</code>	Guarda a lista da tabuada em <code>els.tabuada</code> .
18	<code>passos: \$("#passos"),</code>	Guarda a lista de passos em <code>els.passos</code> . A vírgula no final é permitida e facilita acrescentar itens depois.
19	<code>};</code>	Fecha as chaves abertas na linha 7. O <code>;</code> marca o fim da instrução.
21	<code>let S = null; // estado da divisão atual</code>	Cria a variável S (de "estado"). Ela vai guardar tudo o que o programa precisa lembrar sobre a divisão em andamento. Usamos <code>let</code> (e não <code>const</code>) porque o valor vai mudar. O valor <code>null</code> quer dizer "nada ainda". O texto depois de <code>//</code> é um comentário.

Função começar: preparar a divisão (linhas 23 a 73)

Roda quando você clica em "Começar". Ela confere o que foi digitado, acha o primeiro dividendo parcial e monta o estado inicial.

Nº	Código	Explicação
23	<code>// ----- Início -----</code>	Um comentário que marca o início de uma seção do código.
25	<code>function começar() {</code>	Declara uma função chamada <code>começar</code> . Uma função é um bloco de código com nome, que só roda quando é "chamada". Os parênteses vazios <code>()</code> indicam que ela não recebe informações. A chave <code>{</code> abre o corpo da função.
26	<code>const rawD = els.dividendo.value.trim();</code>	Lê o texto digitado no campo do dividendo (<code>.value</code>) e tira os espaços do começo e do fim (<code>.trim()</code>). O nome <code>rawD</code> quer dizer "D cru", ou seja, ainda não conferido.
27	<code>const rawd = els.divisor.value.trim();</code>	Faz o mesmo com o divisor. Repare que <code>rawD</code> e <code>rawd</code> são nomes diferentes, porque o JavaScript diferencia letras maiúsculas e minúsculas.
29	<code>if (!/^d+\$/ .test(rawD)) return erroInicial("Digite o dividendo usando apenas algarismos (0 a 9).");</code>	Confere se o dividendo tem só algarismos. A expressão regular <code>/^d+\$/</code> é um padrão de texto: <code>^</code> é o início, <code>d</code> é um algarismo, <code>+</code> é "um ou mais" e <code>\$</code> é o fim. Ou seja: "do começo ao fim, só algarismos, pelo menos um". O <code>.test()</code> responde <code>true</code> (verdadeiro) ou <code>false</code> (falso), e o <code>!</code> inverte a resposta. Então, se o texto não for válido, a função mostra o erro e o <code>return</code> encerra a função ali mesmo.
30	<code>if (!/^d+\$/ .test(rawd)) return erroInicial("Digite o divisor usando apenas algarismos (0 a 9).");</code>	A mesma conferência para o divisor.

Nº	Código	Explicação
32	<code>const D = rawD.replace(/^0+(?=\d)/, "");</code>	Tira os zeros à esquerda: 007 vira 7 . A expressão <code>/^0+(?=\d)/</code> encontra zeros no início desde que venha outro algarismo depois (o <code>(?=\d)</code> "olha adiante" sem apagar). Por isso, o número 0 sozinho continua 0 . O resultado, D , continua sendo texto , e assim podemos pegar cada algarismo pela posição.
33	<code>const d = BigInt(rawd);</code>	BigInt transforma o texto do divisor num número inteiro grande e exato . Os números comuns do JavaScript perdem precisão quando passam de uns 9 quatrilhões; com BigInt , isso não acontece.
34	<code>if (d === 0n) return erroInicial("Não existe divisão por zero! Escolha um divisor maior que 0.");</code>	Não existe divisão por zero. O 0n é o número zero no formato BigInt (o n no final indica isso), e o <code>===</code> compara se dois valores são exatamente iguais.
36	<code>els.erroInicial.hidden = true;</code>	Esconde a mensagem de erro, caso ela estivesse aparecendo de uma tentativa anterior.
38	<code>// Primeiro dividendo parcial: o menor começo do dividendo em que o divisor cabe.</code>	Um comentário explicando o próximo trecho: achar o primeiro dividendo parcial.
39	<code>let fim = 0;</code>	fim vai guardar a posição do último algarismo do primeiro dividendo parcial. No JavaScript, as posições começam em 0 : em "7854", o 7 está na posição 0 e o 8 na posição 1.
40	<code>let parcial = 0n;</code>	parcial começa em zero e vai acumulando o valor.
41	<code>for (; fim < D.length; fim++) {</code>	Um laço for : repete o bloco enquanto fim < D.length (enquanto não acabarem os algarismos), somando 1 a fim a cada volta (fim++). A primeira parte do for está vazia porque fim já foi criado na linha 39, já que vamos precisar dele depois do laço.
42	<code>parcial = parcial * 10n + BigInt(D[fim]);</code>	"Acrescenta" o próximo algarismo à direita do número: multiplica por 10 e soma o algarismo. Por exemplo, 7 vira $7 \times 10 + 8 = 78$. O D[fim] pega o caractere que está na posição fim .
43	<code>if (parcial >= d) break;</code>	Se o parcial já é maior ou igual ao divisor, o divisor "cabe" nele. O break interrompe o laço.
44	<code>}</code>	Fecha o laço for aberto na linha 41.
45	<code>if (fim === D.length) fim = D.length - 1;</code>	Se o laço chegou ao fim sem o divisor caber, o dividendo inteiro é menor que o divisor (como em $3 \div 7$). Nesse caso, usamos o dividendo inteiro, e o quociente vai ser 0.
47	<code>S = {</code>	Cria o objeto de estado S . Ele é a "memória" do programa.
48	<code>D,</code>	D , é um atalho para D : D guarda o dividendo (em texto) com o mesmo nome.
49	<code>d,</code>	Guarda o divisor (em BigInt).

Nº	Código	Explicação
50	<pre>linhas: [{ inicio: 0, texto: D, tipo: "dividendo" }],</pre>	linhas é uma lista (um *array* , entre colchetes []) com as linhas desenhadas na conta. Cada linha é um objeto com três informações: inicio (a coluna do primeiro algarismo), texto (os algarismos) e tipo . A primeira linha é o próprio dividendo.
51	<pre> parcial: { linha: 0, de: 0, ate: fim },</pre>	parcial diz onde está o dividendo parcial atual: em qual linha e da coluna de até a coluna ate .
52	<pre> proximo: fim + 1, // próximo algarismo do dividendo a ser baixado</pre>	proximo é a posição do próximo algarismo do dividendo que ainda vai ser "baixado".
53	<pre> baixados: new Set(),</pre>	new Set() cria um conjunto , uma lista que não aceita repetições. Ele guarda as colunas já baixadas, para desenhar a setinha ↓.
54	<pre> quociente: "",</pre>	O quociente começa como um texto vazio e vai ganhando algarismos.
55	<pre> escolha: null, // { k, produto, diferenca, ok }</pre>	escolha vai guardar o algarismo escolhido no momento. Ainda não há nenhum (null).
56	<pre> fase: "escolher", // escolher baixar fim</pre>	fase controla o que aparece no painel. Pode ser "escolher" (escolher um algarismo), "baixar" (baixar o próximo algarismo) ou "fim" (a conta terminou).
57	<pre> resto: null,</pre>	O resto final, que ainda não é conhecido.
58	<pre> passo: 0,</pre>	Um contador de passos.
59	<pre>};</pre>	Fecha as chaves abertas na linha 47. O ; marca o fim da instrução.
61	<pre>els.passos.innerHTML = "";</pre>	Apaga a lista de passos da divisão anterior. innerHTML = "" substitui o conteúdo por "nada".
62	<pre>els.tabuada.innerHTML = "";</pre>	Apaga a tabuada anterior.
63	<pre>for (let k = 0n; k <= 9n; k++) {</pre>	Um laço que repete com k valendo 0, 1, 2... até 9. O n aparece porque k vai ser multiplicado pelo divisor, que é BigInt .
64	<pre> const li = document.createElement("li");</pre>	document.createElement("li") cria um item de lista na memória . Ele ainda não aparece na tela.
65	<pre> li.textContent = `\${d} × \${k} = \${d * k}`;</pre>	Coloca o texto no item. As crases criam um *template string* : tudo o que está dentro de \${ } é calculado e inserido no texto. Por exemplo, com divisor 6 e k igual a 3, o texto fica "6 × 3 = 18".
66	<pre> els.tabuada.appendChild(li);</pre>	appendChild coloca o item dentro da lista da tabuada. A partir daí ele aparece na tela.
67	<pre>}</pre>	Fecha o laço for aberto na linha 63.
69	<pre>els.area.hidden = false;</pre>	Mostra a área de trabalho, tirando o hidden .

Nº	Código	Explicação
70	<code>render();</code>	Chama a função render , que desenha tudo na tela (veja a linha 393).
71	<code>els.area.scrollToView({ behavior: "smooth", block: "start" });</code>	Rola a página suavemente (smooth) até a área de trabalho.
72	<code>}</code>	Fim da função comecar , aberta na linha 25.

Funções erroInicial e exemplo (linhas 74 a 86)

Nº	Código	Explicação
74	<code>function erroInicial(msg) {</code>	Declara a função erroInicial . Ela recebe um parâmetro , msg , que é a mensagem a mostrar.
75	<code>els.erroInicial.textContent = msg;</code>	textContent coloca texto puro no elemento. É uma forma segura: se o texto tiver algo parecido com HTML, aparece como texto e não vira código.
76	<code>els.erroInicial.hidden = false;</code>	Mostra a mensagem de erro.
77	<code>}</code>	Fim da função erroInicial , aberta na linha 74.
79	<code>function exemplo() {</code>	Declara a função exemplo , que sorteia uma conta.
80	<code>const divisor = 2 + Math.floor(Math.random() * 18);</code>	Math.random() sorteia um número decimal entre 0 e 1 (o 1 nunca sai). Multiplicado por 18, dá algo entre 0 e 17,99. Math.floor arredonda para baixo, o que dá de 0 a 17. Somando 2, o divisor fica entre 2 e 19 .
81	<code>const dividendo = divisor * (10 + Math.floor(Math.random() * 990)) + Math.floor(Math.random() * divisor);</code>	Monta um dividendo "amigável": o divisor vezes um número de 10 a 999, mais um resto sorteado menor que o divisor. Assim, a conta sempre tem de 2 a 3 algarismos no quociente, e às vezes tem resto.
82	<code>els.dividendo.value = dividendo;</code>	Escreve o dividendo sorteado no campo.
83	<code>els.divisor.value = divisor;</code>	Escreve o divisor sorteado no campo.
84	<code>comecar();</code>	Chama comecar() , como se você tivesse clicado no botão.
85	<code>}</code>	Fim da função exemplo , aberta na linha 79.

Funções ajudantes (linhas 87 a 108)

Funções pequenas que respondem perguntas usadas em vários lugares.

Nº	Código	Explicação
87	<code>// ----- Ajudantes -----</code>	Um comentário que marca a seção das funções ajudantes.
89	<code>function textoParcial() {</code>	A função textoParcial devolve os algarismos do dividendo parcial atual, em texto.

Nº	Código	Explicação
90	<code>const { linha, de, ate } = S.parcial;</code>	Isto é uma desestruturação : cria de uma vez três constantes (linha , de e ate), copiando as propriedades de mesmo nome de S.parcial .
91	<code>const l = S.linhas[linha];</code>	l é a linha da conta onde está o dividendo parcial.
92	<code>return l.texto.slice(de - l.inicio, ate - l.inicio + 1);</code>	slice(a, b) recorta um pedaço do texto, da posição a até antes da posição b . Subtraímos l.inicio para converter "coluna da grade" em "posição dentro do texto da linha". O + 1 aparece porque o slice não inclui a posição final. O return devolve o resultado para quem chamou a função.
93	<code>}</code>	Fim da função textoParcial , aberta na linha 89.
95–96	<code>function valorParcial() { return BigInt(textoParcial());</code>	valorParcial pega o texto do parcial e o transforma em número (BigInt).
97	<code>}</code>	Fim da função valorParcial , aberta na linha 95.
99–100	<code>function temMaisAlgarismos() { return S.proximo < S.D.length;</code>	temMaisAlgarismos responde true se ainda existem algarismos do dividendo que não foram baixados, ou seja, se a posição do próximo é menor que o tamanho do dividendo.
101	<code>}</code>	Fim da função temMaisAlgarismos , aberta na linha 99.
103	<code>function addPasso(html) {</code>	A função addPasso acrescenta uma frase à lista "Passos". Ela recebe um texto com HTML.
104	<code> const li = document.createElement("li");</code>	Cria um novo item de lista.
105	<code> li.innerHTML = html;</code>	innerHTML , diferente de textContent , interpreta as tags HTML. Usamos para destacar os números com <code> . Aqui é seguro, porque o texto só contém números gerados pelo próprio programa.
106	<code> els.passos.appendChild(li);</code>	Coloca o item na lista de passos.
107	<code>}</code>	Fim da função addPasso , aberta na linha 103.

escolher: quando você escolhe um algarismo (linhas 109 a 138)

Esta é a função chamada toda vez que você muda a caixa de seleção. Ela calcula o produto e a diferença e decide se a escolha está certa, grande demais ou pequena demais.

Nº	Código	Explicação
109	<code>// ----- Ações -----</code>	Um comentário que marca a seção das ações do usuário.
111	<code>function escolher(valor) {</code>	A função escolher recebe valor , que é o texto da opção escolhida: de "0" a "9" , ou "" (vazio) se você escolheu o traço.
112	<code> if (valor === "") {</code>	O if ("se") testa uma condição. Aqui: se o valor for vazio...

Nº	Código	Explicação
113	<code>S.escolha = null;</code>	...apaga a escolha atual...
114	<code>return render();</code>	...redesenha a tela e sai da função.
115	<code>}</code>	Fecha o bloco do <code>if/else</code> aberto na linha 112.
116	<code>const k = BigInt(valor);</code>	<code>k</code> é o algarismo escolhido, transformado em <code>BigInt</code> . Isso é necessário porque o JavaScript não deixa misturar <code>BigInt</code> com números comuns na mesma conta.
117	<code>const P = valorParcial();</code>	<code>P</code> é o valor do dividendo parcial.
118	<code>const produto = S.d * k;</code>	O produto é o divisor vezes o algarismo escolhido.
119	<code>const diferenca = P - produto;</code>	A diferença é o dividendo parcial menos o produto.
120	<code>let ok = true;</code>	A variável <code>ok</code> começa como verdadeira: partimos do princípio de que a escolha está certa.
121	<code>let msg;</code>	<code>msg</code> vai guardar a mensagem. Por enquanto, não tem valor.
123	<code>if (produto > P) {</code>	Primeiro teste: se o produto for maior que o dividendo parcial, a escolha é grande demais , porque não dá para tirar, por exemplo, 24 de 18.
124	<code>ok = false;</code>	Marca a escolha como errada.
125	<code>msg = `\${S.d} × \${k} = \${produto}, que é maior que \${P}. Não dá para tirar \${produto} de \${P}. Escolha um número menor.`;</code>	Monta a mensagem explicando o erro, com os números de verdade inseridos pelo <code>`</code> <code>`</code> .
126	<code>} else if (diferenca >= S.d) {</code>	<code>else if</code> quer dizer "senão, se...". Segundo teste: se a diferença for maior ou igual ao divisor, a escolha é pequena demais , porque o divisor ainda cabe mais uma vez. É a regra da divisão: o que sobra em cada passo tem que ser menor que o divisor.
127	<code>ok = false;</code>	Marca a escolha como errada.
128	<code>msg = `\${S.d} × \${k} = \${produto} e sobriam \${diferenca}, que é maior ou igual ao divisor \${S.d}. Ainda cabe mais: escolha um número maior.`;</code>	Monta a mensagem de "escolha um número maior".
129	<code>} else if (k === 0n) {</code>	Terceiro caso: se passou nos testes e o algarismo é 0, o divisor realmente não cabe, e 0 está certo.
130	<code>msg = `Isso! \${S.d} não cabe em \${P}, então colocamos 0 no quociente.`;</code>	Uma mensagem de acerto para o zero.
131	<code>} else {</code>	<code>else</code> quer dizer "senão", ou seja, todos os outros casos: a escolha está certa e é maior que zero.
132	<code>msg = `Muito bem! \${S.d} × \${k} = \${produto}. Agora vamos subtrair: \${P} - \${produto}.`;</code>	Uma mensagem de acerto, já anunciando a subtração.
133	<code>}</code>	Fecha o bloco do <code>if/else</code> aberto na linha 131.
135	<code>S.escolha = { k, produto, diferenca, ok, msg };</code>	Guarda tudo em <code>S.escolha</code> . <code>{ k, produto, ... }</code> é um atalho para <code>{ k: k, produto: produto, ... }</code> .

Nº	Código	Explicação
136	<code>render();</code>	Redesenha a tela, com a prévia do produto e a mensagem.
137	<code>}</code>	Fim da função <code>escolher</code> , aberta na linha 111.

confirmarZero: colocar 0 no quociente (linhas 139 a 151)

Nº	Código	Explicação
139	<code>function confirmarZero() {</code>	Roda quando você confirma o 0 no quociente.
140	<code>const P = valorParcial();</code>	Pega o valor do dividendo parcial.
141	<code>S.quotiente += "0";</code>	<code>+=</code> acrescenta ao final: o texto do quociente ganha um "0".
142	<code>S.passo++;</code>	<code>++</code> soma 1 ao contador de passos.
143	<code>addPasso(`<code>\${S.d}</code> não cabe em <code>\${P}</code> → quociente recebe <code>0</code>.`);</code>	Registra o passo na lista.
144	<code>S.escolha = null;</code>	Apaga a escolha, porque ela já foi usada.
145	<code>if (temMaisAlgarismos()) {</code>	Se ainda há algarismos para baixar...
146	<code>baixar();</code>	...baixa o próximo imediatamente. No método da chave, quando o divisor não cabe, não se escreve uma subtração de zero.
147	<code>} else {</code>	Senão (não há mais algarismos)...
148	<code>terminar(P);</code>	...termina a conta, e o resto é o próprio dividendo parcial.
149	<code>}</code>	Fecha o bloco do <code>if/else</code> aberto na linha 147.
150	<code>}</code>	Fim da função <code>confirmarZero</code> , aberta na linha 139.

subtrair: escrever o produto e a diferença (linhas 152 a 185)

Nº	Código	Explicação
152	<code>function subtrair(respostaAluno) {</code>	A função <code>subtrair</code> recebe <code>respostaAluno</code> . Esse parâmetro só é usado no modo "eu faço a subtração". No modo automático, a função é chamada sem nada, e o parâmetro fica <code>undefined</code> ("indefinido").
153	<code>const { k, produto, diferenca } = S.escolha;</code>	Desestruturação: copia <code>k</code> , <code>produto</code> e <code>diferenca</code> da escolha atual.
155	<code>if (respostaAluno !== undefined) {</code>	Se veio uma resposta digitada (modo prática)...
156	<code>const txt = respostaAluno.trim();</code>	...tira os espaços dela...
157	<code>if (!/^\\d+\$/.test(txt) BigInt(txt) !== diferenca) {</code>	...e testa se ela não é só algarismos ou (<code> </code>) se o número é diferente (<code>!==</code>) da diferença correta.

Nº	Código	Explicação
158	<code>S.escolha.erroSub = `Ainda não. Confira a conta \${valorParcial()} - \${produto}. Dica: comece pelas unidades.`;</code>	Se errou, guarda uma mensagem com uma dica.
159	<code>return render();</code>	Redesenha a tela e para aqui: a conta não avança até a resposta estar certa.
160	<code>}</code>	Fecha o bloco do <code>if/else</code> aberto na linha 157.
161	<code>}</code>	Fecha o bloco do <code>if/else</code> aberto na linha 155.
163	<code>const P = valorParcial();</code>	Pega o valor do dividendo parcial.
164	<code>const ate = S.parcial.ate;</code>	<code>ate</code> é a coluna do último algarismo do parcial. O produto e a diferença são alinhados pela direita nessa coluna: unidade embaixo de unidade.
165–166	<code>const prodTxt = produto.toString(); const difTxt = diferenca.toString();</code>	<code>toString()</code> transforma os números em texto, para desenhar algarismo por algarismo.
168	<code>S.linhas.push({ inicio: ate - prodTxt.length + 1, texto: prodTxt, tipo: "produto" });</code>	<code>push</code> acrescenta uma linha no final da lista: a linha do produto . A coluna de início é <code>ate - tamanho + 1</code> . Por exemplo, se <code>ate</code> é 1 e o produto é "18" (2 algarismos), o início é $1 - 2 + 1 = 0$. Assim, o 8 cai exatamente na coluna 1.
169	<code>S.linhas.push({ inicio: ate - difTxt.length + 1, texto: difTxt, tipo: "diferenca" });</code>	Acrescenta a linha da diferença , alinhada do mesmo jeito.
170	<code>S.quociente += k.toString();</code>	Acrescenta o algarismo escolhido ao quociente.
171	<code>S.passo++;</code>	Soma 1 ao contador de passos.
172–175	<code>addPasso(`<code>\${S.d}</code> cabe <code>\${   {k}</code> vez(es) em <code>\${P}</code>: ` + `<code>\${S.d} × \${k} =   \${produto}</code> e <code>\${P} - \$   {produto} = \${diferenca}</code>.`);</code>	Registra o passo. O texto foi dividido em duas partes, unidas com <code>+</code> (para textos, o <code>+</code> junta as partes), só para a linha não ficar comprida demais.
176	<code>S.escolha = null;</code>	Apaga a escolha, porque ela já foi usada.
178	<code>if (temMaisAlgarismos()) {</code>	Se ainda há algarismos para baixar...
179	<code>S.fase = "baixar";</code>	...muda a fase para "baixar" , e o painel vai mostrar o botão "Baixar o ...".
180	<code>render();</code>	Redesenha a tela.
181	<code>} else {</code>	Senão...
182	<code>terminar(diferenca);</code>	...termina a conta, e o resto é a última diferença.
183	<code>}</code>	Fecha o bloco do <code>if/else</code> aberto na linha 181.
184	<code>}</code>	Fim da função <code>subtrair</code> , aberta na linha 152.

baixar e terminar (linhas 186 a 207)

Nº	Código	Explicação
186	<code>function baixar() {</code>	A função baixar "desce" o próximo algarismo do dividendo.
187	<code>const col = S.proximo;</code>	col é a posição do algarismo a baixar. Como o dividendo começa na coluna 0, a posição no texto é a mesma coluna da grade.
188	<code>const alg = S.D[col];</code>	alg é o próprio algarismo (um caractere).
189	<code>const idx = S.linhas.length - 1;</code>	idx é a posição da última linha da conta. É length - 1 porque as posições começam em 0.
190	<code>const linha = S.linhas[idx];</code>	linha é essa última linha, que é a diferença que acabou de ser escrita.
192	<code>linha.texto += alg;</code>	Acrescenta o algarismo ao final do texto da linha: "1" vira "18". Como a diferença termina exatamente na coluna anterior, o algarismo novo fica embaixo da posição dele no dividendo, como no caderno.
193	<code>linha.baixados = (linha.baixados 0) + 1;</code>	Conta quantos algarismos foram baixados nesta linha, para pintá-los de outra cor. A expressão (linha.baixados 0) quer dizer "use linha.baixados ou, se ainda não existir, use 0".
194	<code>S.baixados.add(col);</code>	Guarda a coluna no conjunto de baixados, para desenhar a setinha ↓ no dividendo.
195	<code>S.parcial = { linha: idx, de: linha.inicio, ate: col };</code>	O novo dividendo parcial passa a ser essa linha inteira.
196	<code>S.proximo++;</code>	Avança a posição do próximo algarismo.
197	<code>addPasso(`Baixamos o <code>\${alg}</code>. Novo dividendo parcial: <code>\${valorParcial()}</code>.`);</code>	Registra o passo.
198	<code>S.fase = "escolher";</code>	Volta para a fase de escolher um algarismo.
199	<code>render();</code>	Redesenha a tela.
200	<code>}</code>	Fim da função baixar , aberta na linha 186.
202	<code>function terminar(resto) {</code>	A função terminar recebe o resto final.
203	<code>S.resto = resto;</code>	Guarda o resto no estado.
204	<code>S.fase = "fim";</code>	Muda a fase para "fim" .
205	<code>render();</code>	Redesenha a tela, agora com o resultado.
206	<code>}</code>	Fim da função terminar , aberta na linha 202.

celula e desenharConta: desenhando a conta (linhas 208 a 261)

Aqui o estado vira desenho: cada algarismo é um quadradinho posicionado numa grade.

Nº	Código	Explicação
208	// ----- Desenho -----	Um comentário que marca a seção de desenho.
210	function celula(linha, coluna, texto, classes) {	A função celula é uma "fábrica de quadradinhos". Ela recebe a linha, a coluna, o texto (um algarismo ou o sinal de menos) e uma lista de classes.
211	const div = document.createElement("div");	Cria uma <div> na memória.
212	div.className = "cell " + classes.join(" ");	className define as classes do elemento: sempre cell , mais as classes extras. O join(" ") junta a lista com espaços: ["product", "ul"] vira "product ul" .
213	div.style.gridRow = linha + 1;	gridRow diz em qual linha da grade o quadradinho fica. A grade do CSS começa em 1, e nós contamos a partir de 0, por isso o + 1 .
214	div.style.gridColumn = coluna + 2; // +1 da coluna do sinal, +1 porque a grade começa em 1	gridColumn diz a coluna. O + 2 é a soma de +1 pela coluna reservada ao sinal de menos e +1 porque a grade começa em 1.
215	div.textContent = texto;	Escreve o algarismo dentro do quadradinho.
216	return div;	Devolve o quadradinho pronto para quem chamou a função.
217	}	Fim da função celula , aberta na linha 210.
219	function desenharConta() {	desenharConta desenha toda a parte esquerda: o dividendo, os produtos e as diferenças.
220	const frag = document.createDocumentFragment();	Um DocumentFragment é uma "caixa temporária" invisível. Montamos todos os quadradinhos nela e colocamos na página de uma vez , o que é mais rápido e evita que a tela pisque.
221	const mostrandoParcial = S.fase === "escolher";	mostrandoParcial é verdadeiro só na fase de escolher, quando o parcial deve ficar amarelo. O === compara, e o resultado (true ou false) é guardado.
222	const { linha: pl, de: pde, ate: pate } = S.parcial;	Uma desestruturação renomeando : linha: pl quer dizer "pegue linha e chame de pl ". Usamos nomes curtos porque eles se repetem bastante abaixo.
224	// Garante a largura total da grade (dividendo + coluna do sinal).	Um comentário explicando a próxima linha.
225	els.conta.style.gridTemplateColumns = `repeat(\${S.D.length + 1}, var(--cell))`;	Define quantas colunas a grade tem: a quantidade de algarismos do dividendo, mais 1 para o sinal. repeat(n, tamanho) é uma função do CSS escrita aqui pelo JavaScript.
227	S.linhas.forEach((l, r) => {	forEach percorre cada linha da conta. Para cada uma, roda a função seta recebendo l (a linha) e r (a posição dela, que vira o número da linha na grade).
228	const sublinhado = l.tipo === "produto";	Se a linha é um produto, ela leva o traço da subtração.
229	if (l.tipo === "produto") {	Se a linha é um produto...

Nº	Código	Explicação
230	<code>frag.appendChild(celula(r, l.inicio - 1, "-", ["product", "ul"]));</code>	...desenha o sinal "-" na coluna antes do primeiro algarismo (<code>l.inicio - 1</code>), em azul e com o traço.
231	<code>}</code>	Fecha o bloco do if/else aberto na linha 229.
232	<code>const nBaixados = l.baixados 0;</code>	Quantos algarismos foram baixados nesta linha (0 se nenhum).
233	<code>[...l.texto].forEach((ch, i) => {</code>	<code>[...l.texto]</code> usa o operador de espalhamento (<code>...</code>) para transformar o texto numa lista de caracteres: "18" vira <code>["1", "8"]</code> . O forEach percorre cada algarismo (<code>ch</code>) com sua posição (<code>i</code>).
234	<code>const col = l.inicio + i;</code>	Calcula a coluna da grade desse algarismo.
235	<code>const cls = [];</code>	Começa uma lista vazia de classes, para ir preenchendo.
236	<code>if (l.tipo === "produto") cls.push("product");</code>	Se é produto, o algarismo fica azul.
237	<code>if (l.tipo === "diferenca") cls.push(i >= l.texto.length - nBaixados ? "brought" : "diff");</code>	Se é diferença, os últimos algarismos (os baixados) recebem a classe brought , e os outros a classe diff (verde). A forma condição ? a : b é o operador ternário : "se a condição for verdadeira, a ; senão, b ".
238– 239	<code>if (l.tipo === "dividendo") { if (S.baixados.has(col)) cls.push("used", "down");</code>	Se é a linha do dividendo e esta coluna já foi baixada, o algarismo fica cinza (used) e ganha a setinha (down). O método has pergunta se o conjunto contém aquele valor.
240	<code>}</code>	Fecha o bloco do if/else aberto na linha 238.
241	<code>if (sublinhado) cls.push("ul");</code>	Se a linha tem traço, cada algarismo também ganha o traço.
242	<code>if (mostrandoParcial && r === pl && col >= pde && col <= pate) cls.push("partial");</code>	O destaque amarelo só aparece se todas as condições forem verdadeiras (&& quer dizer "e"): estamos na fase de escolher, esta é a linha do parcial e a coluna está entre o início e o fim do parcial.
243	<code>frag.appendChild(celula(r, col, ch, cls));</code>	Cria o quadradinho com tudo isso e o coloca na caixa temporária.
244	<code>});</code>	Fecha as chaves e os parênteses abertos na linha 233. O ; marca o fim da instrução.
245	<code>});</code>	Fecha as chaves e os parênteses abertos na linha 227. O ; marca o fim da instrução.
247	<code>// Prévia do produto escolhido, embaixo do dividendo parcial.</code>	Um comentário explicando o próximo trecho: a prévia do produto.
248	<code>if (S.fase === "escolher" && S.escolha && S.escolha.k > 0n) {</code>	Se estamos escolhendo, já existe uma escolha e ela é maior que zero (o zero não mostra produto)...
249	<code>const r = S.linhas.length;</code>	...a prévia vai na próxima linha livre, abaixo de todas as outras.
250	<code>const txt = S.escolha.produto.toString();</code>	O texto do produto escolhido.

Nº	Código	Explicação
251	<code>const inicio = pate - txt.length + 1;</code>	A coluna de início, alinhada pela direita com o fim do parcial (pate).
252	<code>const extra = S.escolha.ok ? [] :</code> <code>["wrong"];</code>	Se a escolha estiver errada, acrescenta a classe wrong (vermelho). Senão, a lista de extras fica vazia.
253	<code>frag.appendChild(celula(r, inicio - 1,</code> <code>"-", ["preview", "ul", ...extra]));</code>	Desenha o sinal de menos da prévia. O ...extra espalha as classes extras dentro da lista.
254– 255	<code>[...txt].forEach((ch, i) => {</code> <code>frag.appendChild(celula(r, inicio +</code> <code>i, ch, ["preview", "ul", ...extra]));</code>	Desenha cada algarismo da prévia.
256	<code>});</code>	Fecha as chaves e os parênteses abertos na linha 254. O ; marca o fim da instrução.
257	<code>}</code>	Fecha o bloco do if/else aberto na linha 248.
259	<code>els.conta.replaceChildren(frag);</code>	replaceChildren troca todo o conteúdo antigo da grade pelo novo de uma vez.
260	<code>}</code>	Fim da função desenharConta , aberta na linha 219.

desenharChave: o divisor e o quociente (linhas 262 a 289)

Nº	Código	Explicação
262	<code>function desenharChave() {</code>	A função desenharChave desenha o lado direito: o divisor e o quociente.
263	<code>els.divisorBox.replaceChildren(</code>	Troca o conteúdo da caixa do divisor por...
264	<code>...[...S.d.toString()].map((ch) => {</code>	...uma lista de , um para cada algarismo do divisor. O .map transforma cada item de uma lista, aqui de caractere em . O primeiro ... espalha a lista final como argumentos do replaceChildren .
265	<code>const s =</code> <code>document.createElement("span");</code>	Cria o .
266	<code>s.textContent = ch;</code>	Coloca o algarismo dentro dele.
267	<code>return s;</code>	Devolve o , que entra na nova lista.
268	<code>})</code>	Fecha as chaves e os parênteses abertos na linha 264.
269	<code>);</code>	Fecha os parênteses abertos na linha 263. O ; marca o fim da instrução.
271	<code>const spans = [...S.quociente].map((ch)</code> <code>=> {</code>	Faz a mesma transformação com os algarismos do quociente já confirmados e guarda a lista em spans .
272– 274	<code>const s =</code> <code>document.createElement("span");</code> <code>s.textContent = ch;</code> <code>return s;</code>	Para cada algarismo, cria um , coloca o algarismo e o devolve.
275	<code>});</code>	Fecha as chaves e os parênteses abertos na linha 271. O ; marca o fim da instrução.

Nº	Código	Explicação
276	<code>if (S.fase === "escolher") {</code>	Se estamos escolhendo, acrescenta mais um provisório no final do quociente.
277	<code>const s = document.createElement("span");</code>	Cria o provisório.
278	<code>if (S.escolha) {</code>	Se já há uma escolha...
279	<code>s.textContent = S.escolha.k.toString();</code>	...mostra o algoritmo escolhido...
280	<code>s.className = "temp" + (S.escolha.ok ? "" : " wrong");</code>	...com a classe temp , mais wrong se estiver errado.
281	<code>} else {</code>	Senão...
282	<code>s.textContent = "?";</code>	...mostra um ponto de interrogação...
283	<code>s.className = "temp empty";</code>	...com as classes temp e empty .
284	<code>}</code>	Fecha o bloco do if/else aberto na linha 281.
285	<code>spans.push(s);</code>	push acrescenta o provisório no final da lista.
286	<code>}</code>	Fecha o bloco do if/else aberto na linha 276.
287	<code>els.quocienteBox.replaceChildren(...span s);</code>	Coloca todos os na caixa do quociente.
288	<code>}</code>	Fim da função desenharChave , aberta na linha 262.

feedback e botao: pequenas fábricas (linhas 290 a 304)

Nº	Código	Explicação
290	<code>function feedback(msg, tipo) {</code>	A função feedback cria uma mensagem colorida. O parâmetro tipo pode ser good , bad ou info .
291	<code>const p = document.createElement("p");</code>	Cria um parágrafo.
292	<code>p.className = "feedback " + tipo;</code>	Define as classes, por exemplo "feedback good" .
293	<code>p.textContent = msg;</code>	Coloca o texto.
294	<code>return p;</code>	Devolve o parágrafo pronto.
295	<code>}</code>	Fim da função feedback , aberta na linha 290.
297	<code>function botao(texto, onClick, primario = true) {</code>	A função botao cria um botão. O terceiro parâmetro tem um valor padrão : se ninguém informar primario , ele vale true .
298	<code>const b = document.createElement("button");</code>	Cria o elemento <button> .
299	<code>b.textContent = texto;</code>	Coloca o texto do botão.
300	<code>if (primario) b.className = "primary";</code>	Se for principal, recebe a classe primary (azul cheio).

Nº	Código	Explicação
301	<code>b.addEventListener("click", onClick);</code>	<code>addEventListener("click", onClick)</code> faz o botão "escutar" o clique: quando alguém clicar, a função <code>onClick</code> é executada. Em JavaScript, funções podem ser passadas como valor, igual a números e textos.
302	<code>return b;</code>	Devolve o botão pronto.
303	<code>}</code>	Fim da função <code>botao</code> , aberta na linha 297.

desenharPainel: perguntas, mensagens e botões (linhas 305 a 392)

O painel muda conforme a fase: escolher, baixar ou fim.

Nº	Código	Explicação
305	<code>function desenharPainel() {</code>	A função <code>desenharPainel</code> monta o painel de controle.
306	<code>const p = els.painel;</code>	<code>p</code> é só um nome curto para o painel.
307	<code>p.replaceChildren();</code>	<code>replaceChildren()</code> sem nada dentro <code>esvazia</code> o painel.
309	<code>if (S.fase === "escolher") {</code>	Fase "escolher":
310	<code>const P = valorParcial();</code>	Pega o valor do dividendo parcial.
311	<code>const pergunta = document.createElement("p");</code>	Cria o parágrafo da pergunta.
312	<code>pergunta.className = "pergunta";</code>	Dá a ele a classe <code>pergunta</code> .
313	<code>pergunta.textContent = `Quantas vezes o \${S.d} cabe em \${P}?`;</code>	Escreve a pergunta, por exemplo "Quantas vezes o 6 cabe em 45?".
314	<code>p.appendChild(pergunta);</code>	Coloca a pergunta no painel.
316–317	<code>const linha = document.createElement("div"); linha.className = "linha";</code>	Cria uma <code><div></code> com a classe <code>linha</code> .
318–319	<code>const label = document.createElement("label"); label.textContent = "Algarismo do quociente: ";</code>	Cria o rótulo "Algarismo do quociente:".
320	<code>const sel = document.createElement("select");</code>	Cria a caixa de seleção (<code><select></code>).
321	<code>sel.id = "seletorQuociente";</code>	Dá a ela um <code>id</code> , para encontrá-la depois (linha 398).
322	<code>sel.appendChild(new Option("—", ""));</code>	<code>new Option(texto, valor)</code> cria uma opção. A primeira é o traço "—", com valor vazio.
323	<code>for (let k = 0; k <= 9; k++) sel.appendChild(new Option(String(k), String(k)));</code>	Um laço que cria as opções de 0 a 9. <code>String(k)</code> transforma o número em texto.
324	<code>sel.value = S.escolha ? S.escolha.k.toString() : "";</code>	Se já existe uma escolha, ela continua selecionada. Senão, fica no traço. É o operador ternário de novo.
325	<code>sel.addEventListener("change", () => escolher(sel.value));</code>	O coração da interação: quando a seleção muda (evento <code>change</code>), a função <code>escolher</code> é chamada com o valor escolhido.

Nº	Código	Explicação
326	<code>label.appendChild(sel);</code>	Coloca a caixa de seleção dentro do rótulo.
327	<code>linha.appendChild(label);</code>	Coloca o rótulo dentro da linha.
328	<code>p.appendChild(linha);</code>	Coloca a linha no painel.
330	<code>if (S.escolha) {</code>	Se já existe uma escolha...
331	<code> p.appendChild(feedback(S.escolha.msg , S.escolha.ok ? "good" : "bad"));</code>	...mostra a mensagem: verde (good) se estiver certa, vermelha (bad) se não.
333	<code> if (S.escolha.ok) {</code>	Se a escolha estiver certa, mostra os botões para continuar.
334– 335	<code> const bts = document.createElement("div"); bts.className = "buttons";</code>	Cria a caixa dos botões.
336	<code> if (S.escolha.k === 0n) {</code>	Caso 1: a escolha certa é o zero .
337– 339	<code> const txt = temMaisAlgarismos() ? `Colocar 0 e baixar o \$             {S.D[S.proximo]}` : "Colocar 0 e terminar";</code>	O texto do botão depende de haver mais algarismos: "Colocar 0 e baixar o 4" ou "Colocar 0 e terminar". O ternário foi quebrado em três linhas para ficar fácil de ler.
340	<code> bts.appendChild(botao(txt, confirmarZero));</code>	Cria o botão, que chama confirmarZero quando é clicado.
341	<code> p.appendChild(bts);</code>	Coloca os botões no painel.
342	<code> } else if (els.praticar.checked) {</code>	Caso 2: a caixinha "Quero fazer a subtração sozinho" está marcada (checked é verdadeiro).
343– 345	<code> const l2 = document.createElement("div"); l2.className = "linha"; l2.style.marginTop = "0.8rem";</code>	Cria uma linha com uma margem em cima. style.marginTop muda o CSS direto pelo JavaScript.
346	<code> const lab = document.createElement("label");</code>	Cria um rótulo...
347	<code> lab.textContent = `Diferença: \$         {P} - \${S.escolha.produto} = `;</code>	...com a conta a fazer, por exemplo "Diferença: 45 – 42 =".
348	<code> const inp = document.createElement("input");</code>	Cria o campo de digitação.
349	<code> inp.inputMode = "numeric";</code>	Teclado numérico no celular.
350	<code> inp.autocomplete = "off";</code>	Sem sugestões automáticas.
351	<code> inp.addEventListener("keydown", (e) => {</code>	Escuta as teclas pressionadas no campo. O e é o objeto do evento , que traz informações sobre o que aconteceu.
352	<code> if (e.key === "Enter") subtrair(inp.value);</code>	Se a tecla foi Enter (e.key === "Enter"), confere a resposta digitada.
353	<code> });</code>	Fecha as chaves e os parênteses abertos na linha 351. O ; marca o fim da instrução.
354	<code> lab.appendChild(inp);</code>	Coloca o campo dentro do rótulo.
355	<code> l2.appendChild(lab);</code>	Coloca o rótulo na linha.

Nº	Código	Explicação
356	<code>l2.appendChild(botao("Conferir", () => subtrair(inp.value)));</code>	Cria o botão "Conferir", que chama subtrair com o valor digitado.
357	<code>p.appendChild(l2);</code>	Coloca a linha no painel.
358	<code>if (S.escolha.erroSub) p.appendChild(feedback(S.escolha.erroSub, "bad"));</code>	Se a última resposta estava errada, mostra a mensagem de erro.
359	<code>setTimeout(() => inp.focus(), 0);</code>	setTimeout(função, 0) espera o navegador terminar de colocar o campo na tela e só depois coloca o cursor nele (focus), para você já poder digitar.
360	<code>} else {</code>	Caso 3: o modo automático.
361	<code>bts.appendChild(botao(`Subtrair (\${P} - \${S.escolha.produto})`, () => subtrair()));</code>	Cria o botão "Subtrair (45 - 42)". Ele chama subtrair() sem argumento, e por isso a resposta não é conferida.
362	<code>p.appendChild(bts);</code>	Coloca os botões no painel.
363	<code>}</code>	Fecha o bloco do if/else aberto na linha 360.
364	<code>}</code>	Fecha o bloco do if/else aberto na linha 333.
365	<code>} else {</code>	Se ainda não existe uma escolha...
366- 368	<code>p.appendChild(feedback("Escolha um número. O produto do divisor pelo número aparece embaixo do dividendo.", "info"));</code>	...mostra uma mensagem de instrução (info).
369	<code>}</code>	Fecha o bloco do if/else aberto na linha 365.
370	<code>} else if (S.fase === "baixar") {</code>	Fase "baixar":
371- 374	<code>const pergunta = document.createElement("p"); pergunta.className = "pergunta"; pergunta.textContent = `Ainda há algarismos no dividendo. Baixe o próximo: \${S.D[S.proximo]}`; p.appendChild(pergunta);</code>	Cria e mostra a pergunta "Ainda há algarismos no dividendo. Baixe o próximo: 5".
375- 376	<code>const bts = document.createElement("div"); bts.className = "buttons";</code>	Cria a caixa dos botões.
377	<code>bts.appendChild(botao(`Baixar o \$ {S.D[S.proximo]} ↓`, baixar));</code>	Cria o botão "Baixar o 5 ↓", que chama baixar quando é clicado.
378	<code>p.appendChild(bts);</code>	Coloca os botões no painel.
379	<code>} else if (S.fase === "fim") {</code>	Fase "fim":
380	<code>const q = BigInt(S.quociente);</code>	Transforma o texto do quociente em número.
381	<code>const r = S.resto;</code>	Pega o resto.
382- 383	<code>const div = document.createElement("div"); div.className = "resultado";</code>	Cria a caixa do resultado.

Nº	Código	Explicação
384-388	<pre> div.innerHTML = `<p class="pergunta">Divisão concluída! 🧮</p>` + `<p>Quociente: <strong>\${q}</strong> &nbsp;</p> Resto: <strong>\${r}</strong>         ` + `(<strong>\${r}</strong> === 0n ? "divisão exata" : "divisão não exata")</p>` + `<p>Prova real: <strong>\${S.d} × \$ {q} + \${r} = \${S.d * q + r}</strong>         ✓</p>`; </pre>	Monta o HTML do resultado juntando vários textos com +. Ele mostra o quociente, o resto, se a divisão é exata (resto igual a zero) ou não, e a prova real: divisor × quociente + resto. A conta $\{S.d \times q + r\}$ é feita de verdade e tem que dar o dividendo. O &nbsp; é um espaço que o HTML nunca apaga nem quebra de linha, e o deixa o texto em negrito.
389	<pre> p.appendChild(div); </pre>	Coloca o resultado no painel.
390	<pre> } </pre>	Fecha o bloco do if/else aberto na linha 379.
391	<pre> } </pre>	Fim da função desenharPainel , aberta na linha 305.

render: redesenhar tudo (linhas 393 a 402)

Nº	Código	Explicação
393	<pre>function render() {</pre>	render é a função central. Ela redesenha a tela inteira a partir do estado S . Essa é uma ideia importante em programação de interfaces: os dados mandam, e a tela é só um reflexo deles. Toda ação muda o estado e chama render() .
394	<pre> desenharConta();</pre>	Desenha a conta.
395	<pre> desenharChave();</pre>	Desenha a chave.
396	<pre> desenharPainel();</pre>	Desenha o painel.
397	<pre> if (S.fase === "escolher") {</pre>	Se estamos escolhendo...
398	<pre> const sel = \$("#seletorQuociente");</pre>	...busca a caixa de seleção pelo id ...
399	<pre> if (sel && !S.escolha) sel.focus({ preventScroll: true });</pre>	...e, se ela existir e ainda não houver escolha, coloca o foco nela. Assim, dá para escolher usando o teclado. O preventScroll: true evita que a tela pule.
400	<pre> }</pre>	Fecha o bloco do if/else aberto na linha 397.
401	<pre>}</pre>	Fim da função render , aberta na linha 393.

Ligando os botões (eventos) (linhas 403 a 420)

Por último, conectamos os botões e campos às funções. Sem isso, clicar não faria nada.

Nº	Código	Explicação
403	// ----- Eventos -----	Um comentário que marca a seção de eventos.
405	<code>\$("#btnComecar").addEventListener("click", comeocar);</code>	Liga o botão "Começar" à função comeocar . Repare que escrevemos comeocar sem parênteses : estamos entregando a função para ser usada depois do clique, e não executando a função agora.

Nº	Código	Explicação
406	<code>\$("#btnExemplo").addEventListener("click", exemplo);</code>	Liga o botão "Exemplo aleatório" à função exemplo .
407	<code>\$ ("#btnRecomecar").addEventListener("click" , () => {</code>	Liga o botão "Nova divisão" a uma função escrita ali mesmo (uma função anônima, sem nome), que faz várias coisas:
408	<code>els.area.hidden = true;</code>	Esconde a área de trabalho.
409	<code>S = null;</code>	Apaga o estado.
410– 411	<code>els.dividendo.value = ""; els.divisor.value = "";</code>	Limpa os dois campos.
412	<code>els.dividendo.focus();</code>	Coloca o cursor no campo do dividendo.
413	<code> window.scrollTo({ top: 0, behavior: "smooth" });</code>	Rola a página até o topo, suavemente.
414	<code>});</code>	Fecha as chaves e os parênteses abertos na linha 407. O ; marca o fim da instrução.
415	<code>[els.dividendo, els.divisor].forEach((inp) =></code>	Para cada um dos dois campos (dividendo e divisor)...
416	<code> inp.addEventListener("keydown", (e) => {</code>	...escuta as teclas pressionadas...
417	<code> if (e.key === "Enter") começar();</code>	...e, se for Enter, começa a divisão. Assim, você não precisa clicar no botão.
418	<code> })</code>	Fecha as chaves e os parênteses abertos na linha 416.
419	<code>});</code>	Fecha os parênteses abertos na linha 415. O ; marca o fim da instrução.
420	<code>els.praticar.addEventListener("change", () => S && render());</code>	Quando a caixinha "praticar" é marcada ou desmarcada, redesenha a tela, mas só se já existir uma divisão em andamento. O S && render() usa o "curto-circuito" do && : se S for null , o render() nem é executado.

Parte 6: Mãos à obra

Como testar suas mudanças

Você não precisa instalar nada complicado para mexer no site:

- 1. Baixe o código: no GitHub, abra o repositório, clique em **Code** e depois em **Download ZIP**. Descompacte o arquivo numa pasta.
- 2. Instale um editor de código gratuito. O mais usado é o **Visual Studio Code** (code.visualstudio.com). Abra a pasta nele.
- 3. Dê dois cliques no arquivo `index.html`, e ele abre no navegador.
- 4. Mude algo no código, salve (Ctrl+S) e recarregue o navegador (F5).
- 5. Se algo não funcionar, aperte **F12** no navegador e abra a aba **Console**: os erros do JavaScript aparecem ali, com o número da linha.

Dica: dentro do script, você pode escrever `console.log(S);` em qualquer função para ver o estado no Console. É a forma mais simples de "espionar" o que o programa está pensando.

Exercícios

Os exercícios estão em ordem de dificuldade. Tente sozinho antes de ler a dica.

Exercício 1: trocar a cor principal (fácil)

Faça os botões e o quociente ficarem verdes, ou de outra cor qualquer.

Dica: mude só uma linha do `style.css`, a variável `--accent` (linha 7). Experimente `#8e44ad` (roxo).

Exercício 2: mudar o título (fácil)

Troque o título da página para "Minha Calculadora de Divisão".

Dica: o título visível está na linha 15 do `index.html`, e o título da aba está na linha 6.

Exercício 3: quadradinhos maiores (fácil)

Deixe os quadradinhos da folha maiores no computador.

Dica: na linha 14 do `style.css`, aumente o último valor do `clamp`, por exemplo de `2.6rem` para `3.2rem`.

Exercício 4: exemplos com divisores maiores (médio)

Faça o botão "Exemplo aleatório" sortear divisores de 2 até 99.

Dica: na linha 80 do `script.js`, troque o `18` por `98`. Por quê? `Math.random() * 98` dá de 0 até 97,99; com `Math.floor` fica de 0 a 97; somando 2, fica de 2 a 99.

Exercício 5: contar os erros (médio)

Mostre no final quantas vezes o usuário escolheu um número errado.

Dica:

- Na função `comecar`, acrescente `erros: 0`, dentro do objeto `S` (perto da linha 58).
- Na função `escolher`, logo depois de cada `ok = false;` (linhas 124 e 127), acrescente `S.erros++;`.
- Na fase `"fim"` do `desenharPainel` (perto da linha 388), acrescente mais um pedaço ao texto do resultado, logo antes do ponto e vírgula da linha 388, por exemplo `+ "<p>Erros: " + S.erros + "</p>"`.

Exercício 6: limite de algarismos (médio)

Hoje o divisor pode ter até 6 algarismos. Limite-o a 3.

Dica: existem dois lugares possíveis. Um é o atributo `maxLength` do campo, na linha 27 do `index.html`. Para ficar ainda mais seguro, crie também uma conferência no `comecar`, no estilo da linha 30, usando `rawd.length > 3`.

Exercício 7: desafio (difícil)

Crie um botão "Desfazer", que volta um passo.

Dica: guarde uma cópia do estado antes de cada mudança, numa lista:

`historico.push(structuredClone(S))`. Para desfazer, use `S = historico.pop(); render();`. O `structuredClone` cria uma cópia completa de um objeto. Pense também em quais lugares precisam guardar o histórico (`subtrair`, `baixar`, `confirmarZero`).

Próximos passos de estudo

Bons materiais gratuitos e em português:

- **MDN Web Docs** (developer.mozilla.org/pt-BR): a referência mais confiável sobre HTML, CSS e JavaScript. Quando quiser saber o que um comando faz, procure aqui.
- **Curso em Vídeo** (cursoemvideo.com), do professor Gustavo Guanabara: cursos completos e gratuitos de HTML5 e CSS3 e de JavaScript, para iniciantes.
- **freeCodeCamp** (freecodecamp.org): exercícios práticos no próprio navegador, com certificado gratuito.

Uma sugestão de caminho:

- 1. HTML e CSS básicos: faça uma página pessoal simples.
- 2. JavaScript básico: variáveis, if, for e funções. Faça pequenos programas no Console.
- 3. DOM e eventos: faça uma calculadora simples de soma, com dois campos e um botão.
- 4. Volte a este código e tente refazer uma parte dele sozinho, por exemplo uma versão para **multiplicação**.

Glossário

Termo	Significado
Algoritmo	Uma sequência de passos para resolver um problema, como a divisão no caderno.
Array (lista)	Vários valores em ordem, entre colchetes: [1, 2, 3]. A primeira posição é a 0.
Atributo	Uma informação extra numa tag HTML, como <code>id="divisor"</code> .
BigInt	Um tipo de número inteiro do JavaScript que faz contas exatas com números enormes.
Booleano	Um valor que só pode ser <code>true</code> (verdadeiro) ou <code>false</code> (falso).
Classe (CSS)	Uma etiqueta que vários elementos podem ter, usada pelo CSS para aplicar estilos.
Comentário	Um texto no código que o computador ignora. Em JS usa <code>//</code> , em CSS usa <code>/* */</code> .
DOM	A página transformada em objetos que o JavaScript pode ler e alterar.
Estado	Os dados que o programa precisa lembrar. Aqui, o objeto <code>S</code> .
Evento	Algo que acontece na página: um clique, uma tecla, uma mudança de seleção.
Expressão regular	Um padrão para conferir textos, como <code>/^d+\$/</code> ("só algarismos").
Função	Um bloco de código com nome que pode ser chamado quando necessário.
id	O nome único de um elemento HTML.

Termo	Significado
Laço (loop)	Um trecho que se repete, como o for.
Objeto	Um conjunto de valores com nome: { nome: valor }.
Parâmetro	Uma informação que uma função recebe, entre parênteses.
Seletor	A parte da regra CSS que diz a quais elementos ela se aplica.
Tag	Um elemento do HTML, escrito entre < e >.
Template string	Um texto entre crases que aceita valores dentro de \${ }.
Variável	Uma "caixa" com nome que guarda um valor (const ou let).